Publicly Verifiable Generalized Secret Sharing Schemes and Their Applications

No Author Given

No Institute Given

Abstract. Generalized secret sharing (GSS) enables flexible access control in distributed systems by allowing secrets to be shared across arbitrary monotone access structures. However, its adoption in transparent and trustless environments is hindered due to the reliance on trusted participants and secure communication channels. This reliance restricts GSS's ability to provide flexible control in the presence of adversaries. In this paper, we propose publicly verifiable generalized secret sharing (PVGSS), a scheme that allows to distribute a secret using generalized access structures while enabling non-interactive public verifiability of participant honesty. PVGSS scheme offers significant potential to advance the fields of blockchain and applied cryptography, such as attribute-based cryptosystem and on-chain secret escrow. Intuitively, we first build two GSS schemes by leveraging recursive Shamir secret sharing and linear secret sharing scheme. Then, we encrypt GSS shares and generate the corresponding non-interactive zero-knowledge proofs. Furthermore, we innovatively apply GSS testing algorithm to show all encrypted shares bind to the dealer's secret, with only $O(|\mathbb{A}|)$ verification complexity, where $|\mathbb{A}|$ is the number of leaf nodes in the access structure $\mathbb{A}$. To exemplify the practical applicability, we implement a decentralized exchange (DEX) protocol, where fairness and accountable arbitration are considered. Our benchmarks on the BN128 curve demonstrate the computational efficiency of PVGSS schemes, while Ethereum gas cost analysis confirms the viability of the DEX implementation.

Keywords: Access structure, Public verifiability, PVGSS, GSS Testing, Decentralized exchange

1 Introduction

Secret sharing (SS) [**?**] and its subsequent extensions, including threshold cryptography [**?**] and distributed key generation (DKG) [**?**], constitute a foundational primitive for quorum-based trust in distributed systems. To ensure correctness of the sharing process, verifiable secret sharing (VSS) [**?**,**?**,**?**] and publicly verifiable secret sharing (PVSS) [**?**,**?**,**?**] were introduced, enabling participants or even external observers to verify that a dealer has distributed consistent shares.

Despite substantial progress, existing secret sharing paradigms remain ill-suited for modern decentralized systems. Emerging applications—such as blockchain,

Web 3.0 [?], and decentralized finance—demand expressive access control beyond simple threshold policies, enabling hierarchical, weighted, or composite authorization. Meanwhile, the trustless nature of these systems requires non-interactive public verifiability, so that any external observer can validate correctness without private channels or trusted coordination. Reconciling expressiveness with public verifiability thus constitutes a fundamental and unresolved challenge, rather than an incremental extension of existing techniques.

Generalized secret sharing (GSS) [?,?,?] addresses the expressiveness gap by supporting arbitrary monotone access structures. However, classical GSS constructions assume private communication channels and do not provide mechanisms for public verifiability. Conversely, PVSS schemes [?,?,?] achieve public verifiability but are inherently restricted to threshold access structures. Ciphertext-policy attribute-based encryption (CPABE) [?,?] supports expressive access policies, yet lacks built-in guarantees that encryptors or authorities behave honestly, rendering accountability external to the primitive. The linear secret sharing scheme (LSSS) [?,?,?] forms the foundational tool enabling expressive mechanisms in many CPABE schemes.

In this work, we initiate the study of publicly verifiable generalized secret sharing (PVGSS), which simultaneously supports generalized access structures and non-interactive public verifiability. A PVGSS scheme satisfies two core properties. *Expressiveness* enables secrets to be protected under arbitrary monotone access policies, thereby advancing research on attribute-based encryption [?,?] in trustless environments. *Public verifiability* ensures that both dealers and shareholders can be held non-interactively accountable by any verifier, making PVGSS particularly well-suited for blockchain settings, where on-chain secret management or governance [?,?,?], scaling [?] and chain interoperability [?,?] impose stringent requirements. To enhance clarity and emphasize the necessity of PVGSS, we present a comparison with existing SS-based schemes in Table ??.

Crucially, PVGSS cannot be obtained by naively combining existing PVSS schemes. The core technical challenge of PVGSS is to prove, in a publicly verifiable manner, that a collection of encrypted shares $\{C_i\}$ is globally consistent with a single underlying secret under an arbitrary access structure. Suppose the access structure is represented as a tree, the secret is associated with the root while encrypted shares are assigned only to the leaf nodes, rendering any straightforward combination of multiple PVSS instances ineffective. To overcome this challenge, we build PVGSS schemes atop the GSS framework, instantiated using either recursive Shamir secret sharing or linear secret sharing schemes (LSSS). Each GSS share s_i is encrypted under the public key of shareholder P_i and is accompanied by non-interactive zero-knowledge (NIZK) proofs derived from Sigma protocols [?] via the Fiat–Shamir heuristic [?]. Furthermore, we propose a verification algorithm (i.e., GSS testing) to enable validity checks for all shares over an arbitrary access structure. With this algorithm, we establish a binding relation between all encrypted shares and the dealer's secret, thereby elegantly resolving the challenge of public verifiability in secret sharing over general access structures.

Table 1: Comparison of SS-related cryptographic primitives

Scheme	Accountable dealer (or encryptor)	Accountable shareholders (or authorities)	Insecure channel	Public verifiability	Expressiveness
SS	✗	✗	✗	✗	✗
VSS	✓	✗	✗	✗	✗
PVSS	✓	✓	✓	✓	✗
CPABE	✗	✗	✓	✗	✓
LSSS	✗	✗	✗	✗	✓
GSS	✗	✗	✗	✗	✓
PVGSS	✓	✓	✓	✓	✓

- Accountability indicates whether misbehavior by dealers (or encryptors) and shareholders (or authorities) can be detected.
- Insecure channel denotes that secret shares may be distributed over public channels.
- Public verifiability means that correctness can be checked by any external party.
- Expressiveness indicates native support for arbitrary monotone access structures.

1.1 Applications

PVGSS introduces a cryptographic primitive that simultaneously supports expressive access control and non-interactive public accountability—two properties that rarely coexist in prior systems. This combination enables new capabilities across several security-critical domains where correctness must be verifiable by untrusted observers and where access policies extend far beyond simple thresholds. In Section **??**, we will dive deep into a decentralized exchange (DEX) protocol applying PVGSS.

Large-scale beacon protocols. Existing PVSS-based distributed randomness beacons [**?**] protocols are fundamentally constrained by threshold policies, limiting committee flexibility and preventing hierarchical or weighted participation. PVGSS enables publicly verifiable share consistency under arbitrary monotone structures, allowing beacon protocols to incorporate multi-tier committees, heterogeneous trust assumptions, and application-specific quorum rules without sacrificing transparency. This expands the design space for scalable, decentralized randomness generation. For example, by partitioning n distributed parties into subgroups of size c, one can define an access control policy of the form $(d \text{ of } (\frac{c}{3} \text{ of } P_1, \ldots, P_c), (\frac{c}{3} \text{ of } P_{c+1}, \ldots, P_{2c}), \ldots, (\frac{c}{3} \text{ of } P_{(\frac{n}{c}-1)c+1}, \ldots, P_n))$, where the parameter d is introduced to balance unpredictability and liveness of the beacon [**?**,**?**]. Under this policy, each party communicates only with a small subgroup of peers, thereby improving scalability.

Attribute-based cryptosystem. PVGSS can significantly enhance attribute-based cryptosystems [**?**,**?**] by ensuring trust and transparency in key distribution and

access control. By replacing LSSS with a PVGSS, a CPABE scheme can be directly transformed into one that is secure against chosen-ciphertext attacks. Moreover, PVGSS enables CPABE encryptors to outsource encryption to untrusted servers while ensuring public accountability for their behaviors.

Blockchain ecosystems. Blockchain environments require protocols that operate over public channels, tolerate adversarial participants, and provide transparent correctness guarantees. PVGSS naturally applies to the scenarios below:

• PVGSS, functioning as a timed commitment [**?**], can be effectively used in on-chain **secret escrow** systems [**?**,**?**,**?**] to provide fine-grained and verifiable management of witnesses, thereby facilitating practical deployments of e-voting and auction protocols.

• In blockchain **bridges** [**?**], PVGSS ensures that the secret values used to lock and release assets are verifiably shared, thereby coordinating data synchronization across multiple validators or blockchains under a single access control policy.

• In **relay networks** [**?**], PVGSS enables secure forwarding of state proofs with public verifiability among different groups or committees.

• In payment or state **channels** [**?**], PVGSS supports flexible quorum policies for dispute resolution, allowing participants to reconstruct secrets only under authorized conditions.

• Incorporating PVGSS into **oracle** [**?**] design strengthens functionality by ensuring decentralized, publicly verifiable data feeds and enabling flexible privilege weighting among oracle nodes.

• By designing robust access control structures, KYC/AML (i.e., the compliance measurements of know your customer/ anti money laundering) can be effectively enforced as required by **stablecoin governance**. For example, an access policy such as (3 of ($Alice$, Bob, (t of $Reg_1, \cdots, Reg_n$))) enables a threshold number of regulators to transparently monitor the transaction between exchangers (i.e., Alice and Bob).

1.2 Contributions

• We introduce Publicly Verifiable General Secret Sharing (PVGSS)–the first secret sharing framework that simultaneously supports arbitrary monotone access structures and non-interactive public verifiability. This resolves a long-standing tension between the expressiveness of general secret sharing (GSS) and the accountability of publicly verifiable schemes.

• We present two PVGSS instantiations: one built atop recursive Shamir secret sharing and the other derived from linear secret sharing schemes (LSSS). First, the individual encrypted shares are validated using Sigma-protocol–based NIZK proofs. Furthermore, we establish a binding relationship between the encrypted shares and the dealer's secret by means of a proposed GSS testing algorithm, which relies exclusively on existing NIZK proof transcripts as input.

• Apart from correctness and verifiability, we formalize the security notions of IND1-secrecy and public verifiability for PVGSS, and rigorously prove that our

constructions are secure under the standard Decisional Diffie-Hellman (DDH) assumption in the random oracle model.

• To validate the practicality of PVGSS, we design and implement a novel DEX protocol that achieves optimistic fairness, stateless arbitration, and on-chain accountability. Our Ethereum-compatible prototype executes swaps for less than $0.55 USD in a 9-arbitrageur setting in the optimistic case, demonstrating strong real-world viability.

2 Related Works

2.1 Access Structures in Secret Sharing

As is well known, traditional SS, VSS, and PVSS protocols [?,?,?,?,?] are designed around the threshold access structure (TAS). Benaloh et al. [?] introduced the concept of generalized secret sharing (GSS) based on recursive threshold secret sharing, where generalized access structure (GAS) is inherently implied. Beyond TAS- or GAS- based schemes, some other protocols have explored specific types of access control mechanisms.

In particular, Simmons [?] points out that real-world applications require more versatile capabilities for secret sharing. In weighted secret sharing (WSS) [?,?], each participant is assigned a positive weight. Only if the sum of weights of some participants exceeds the predefined threshold value, can the secret be reconstructed.

In multipartite secret sharing (MSS) [?,?], an access structure defined over n participants is partitioned into disjoint $m(\ll n)$ compartments, where participants within the same compartment have equal weights. In MSS, each compartment must achieve a specific internal agreement before contributing to the reconstruction of the secret. Tassa et al. [?] point out that any GAS can be regarded as a multipartite access structure with singleton compartments. Hence, reconstructing the secret in MSS scheme depends only on the number of participants from each compartment, rather than their individual identities. In this way, practical applications can focus on m compartments, where TAS is applied, instead of n individuals. However, participants may be designed in different access structures in a single system, resulting in different divisions of disjoint compartments.

Hierarchical secret sharing (HSS) [?,?], also referred as multilevel secret sharing [?,?], places participants in disjoint levels according to their importance. HSS can be applied in managing staff in big companies, where employees are assigned to different levels with different privileges. HSS is categorized into disjunctive and conjunctive types [?]. Further, Drăgan et al. [?] introduce the distributive weighted threshold secret sharing, where participants at the same level have the same weight, and the threshold values are 1.

Other related works, such as proactive secret sharing [?] and multiple secret sharing [?], can be regarded as extensions of SS. Moreover, these concepts can also be incorporated into our GSS and PVGSS schemes.

2.2 (Publicly) Verifiable Secret Sharing

Dealer in the SS scheme may distribute invalid shares to deviate from the protocol. To mitigate this problem, VSS [?,?] ensures that participants can verify the validity of the shares distributed by the dealer, while PVSS [?,?] extends this by allowing any external observer to verify the integrity of the shares. Notably, Galil and Yung [?] combine VSS, partition encryption and NIZK proofs to design a more efficient and fully polynomial scheme known as partitioned VSS.

The first VSS scheme, proposed by Feldman [?], enhances Shamir secret sharing by adding public commitments to ensure share validity. Shareholders verify their shares against these commitments, ensuring the dealer's honesty without revealing the secret. Recently, polynomial commitments [?,?,?] are widely adopted to design new VSS schemes. The secret shares of VSS should not be transferred publicly.

Stadler [?] put forward the first PVSS scheme in 1996. Intuitively, PVSS can be achieved by integrating public key encryption into VSS schemes, allowing secret shares to be delivered and verified on public channels. For example, ElGamal, RSA, and Pailler cryptosystems are utilized in [?,?], [?], and [?], respectively. Early PVSS schemes require $O(nt)$ verification complexity, where n and t are the amount of shareholders and threshold value, respectively. In 2017, Cascudo et al. [?] introduced SCRAPE PVSS which for the first time achieves $O(n)$ verification complexity. As the core technique, Reed-Solomon code is employed to make all encrypted shares be verified in a linear operation.

Recently, Cascudo et al. successively proposed ALBATROSS [?], HEPVSS/DHPVSS [?], and qCLPVSS [?], which all maintain $O(n)$ verification complexity. ALBATROSS [?] leverages low-degree exponent interpolation to generate NIZK proof, which is essentially acquired from Sigma protocol and Fiat-Shamir heuristic. HEPVSS[?] also uses ElGamal to encrypt the secret share, while DHPVSS[?] leverages Shamir secret sharing on group $\mathbb{G}$ to hide shares. qCLPVSS [?] enables users to recover the original secret $s \in \mathbb{Z}_q$ by leveraging class group, making it applicable in more wider applications, such as distributed key generation [?,?] and MPC [?,?] protocols.

3 Preliminaries

In this paper, $\mathbb{Z}_q$ denotes a finite field with prime order q, and $\mathbb{G}$ represents a cyclic group of the same order generated by g.

3.1 Generalized Access Structure (GAS)

Definition 1 (GAS [?,?]). *Let $\mathcal{P} = \{P_1, \cdots, P_n\}$ be a set of parties. A collection $\mathbb{A} \in 2^{\mathcal{P}}$ is a generalized access structure (GAS, commonly abbreviated as AS in previous research [?,?]), if it meets the following two conditions:*

- *Non-triviality: if $B \in \mathbb{A}$, then $|B| \neq 0$.*
- *Monotonicity: if $B \in \mathbb{A}$, $B \subseteq C$, then $C \in \mathbb{A}$.*

If $B \in \mathbb{A}$, we say $\mathbb{A}$ is satisfied by B or B is an authorized set; Otherwise, $\mathbb{A}$ is not satisfied by B or B is unauthorized.

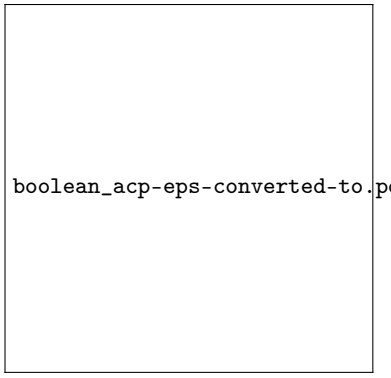

Fig. 1: An access structure using boolean formula, where $\{P_{1,1}, P_{2,1}\}$ and $\{P_{2,*}, P_{3,1}\}$ are authorized sets.

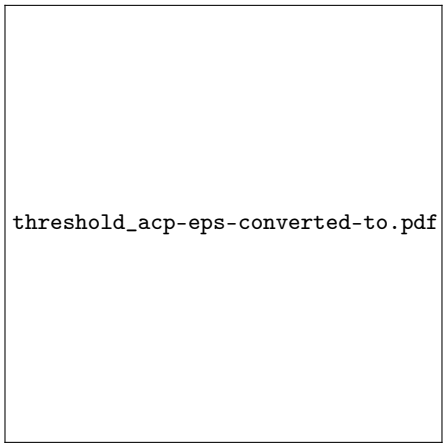

Fig. 2: An access structure using threshold gate, where $\{P_{3,1}, P_{2,*}\}$ and $\{P_{1,1}, P_{3,1}, P_{4,1}\}$ are authorized sets.

For example, $(P_{1,1} \wedge P_{2,1}) \vee (P_{2,2} \wedge P_{3,1})$ and $2\ of\ (2\ of\ (P_{1,1}, P_{2,1}, P_{3,1}), (1\ of\ (P_{1,2}, P_{4,1})))$ are access structures, as shown by Figure **??** and Figure **??**, where $P_{i,j}$ represents the jth authorization granted to party P_i and $*$ is a wildcard. The disjunction ($\vee$) and conjunction ($\wedge$) in a boolean formula based access structure (cf. Figure **??**) is equal to the gate value of 1 and 2 in a threshold-gate based access structure (cf. Figure **??**), respectively. Hence, we only consider threshold-gate based access structure in this paper.

Below, we present several of our observations and summarize them as corollaries. These corollaries can be derived by following the definition of GAS.

Corollary 1 *The boolean or threshold-gates based access structure can be regarded as a proposition, which outputs* true *given an authorized set and outputs* false *otherwise.* □

Corollary 2 *An access structure $\mathbb{A}$ can be regarded as a recursive data structure, meaning that any node x in $\mathbb{A}$ with its children nodes is also an access structure, denoted by $\mathbb{A}_x$. For example, there are 8 sub access structures in $\mathbb{A}$ of Figure* **??**. □

Corollary 3 *If $B \in \mathbb{A}$, there exists a path (i.e., a leaf node to the root node) in which each sub access structure is satisfied by B. We call it a satisfied path.* □

3.2 Shamir Secret Sharing (SS)

Definition 2 (Shamir SS). *Shamir secret sharing (SS) scheme allows a dealer to split a secret into n shares for n shareholders. With only a threshold t ($\leq n$) shares, the secret can be successfully recovered, while fewer than t shares reveal nothing about the secret. Shamir SS is defined with following two algorithms.*

- $\{s_1, s_2, \cdots, s_n\} \leftarrow \mathsf{Share}(s, n, t)$*: Inputs a secret $s \in \mathbb{Z}_q$, n and t, chooses a polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Finally, the algorithm outputs n shares $f(i)$, $i \in [1, n]$.*
- $s \leftarrow \mathsf{Recon}(Q)$*: Inputs any t shares $Q \subseteq \{s_1, s_2, \cdots, s_n\}$, reconstructs the secret s via the Lagrange interpolation.*

Definition 3 (Shamir SS on Group $\mathbb{G}$). *Definition* **??** *has introduced the original Shamir secret sharing scheme, where a secret $s \in \mathbb{Z}_q$ is shared and recovered. Here we consider how to share secret $S \in \mathbb{G}$ and the dealer does not know the value s such that $S = g^s$. The Shamir secret sharing on a group of order q is defined as below:*

- $\{S_1, S_2, \cdots, S_n\} \leftarrow \mathsf{GrpShare}(S, n, t)$*: Inputs a secret $S \in \mathbb{G}$, n and t, chooses a polynomial $f(x) = \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Set $S_i = S \cdot g^{f(i)}$, $i \in [0, n]$. Obviously, $S_0 = S$.*
- $S \leftarrow \mathsf{GrpRecon}(Q)$*: Inputs any t shares Q, reconstructs the secret $S \leftarrow S_0 = \prod_{i \in I} S_i^{\prod_{j \in I, j \neq i} \frac{-j}{i-j}}$, where I is the set containing indexes of the shares Q.*

Shamir SS or Shamir SS on Group $\mathbb{G}$ should satisfy:

• **Correctness**: If the secret sharing process is correctly followed, any authorized set of participants can reconstruct the secret.

• **Secrecy**: Nothing information about the secret is revealed, given any unauthorized set of participants.

3.3 Linear Secret Sharing Scheme (LSSS)

Definition 4 (LSSS [?,?,?]). *Let $\mathcal{P}$ be a set of shareholders. A linear secret sharing scheme over a ring $\mathbb{Z}_q$ for an access structure $\mathbb{A}$ is a tuple (M, ρ), where $M \in \mathbb{Z}_q^{|\mathbb{A}| \times l}$ is a share-generating matrix with $|\mathbb{A}|$ rows and l columns* [1] *and ρ is a row-labeling function. The scheme consists of the following two algorithms:*

- $\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\} \leftarrow \mathsf{LSSSShare}(s, \mathbb{A})$*: To share a value $s \in \mathbb{Z}_q$ under access structure $\mathbb{A}$, randomly choose $v_2, \ldots, v_{|\mathbb{A}|} \xleftarrow{R} \mathbb{Z}_q$ and set $\boldsymbol{v} = [s, v_2, \ldots, v_l]^\top$. Then, $[s_1, \cdots, s_{|\mathbb{A}|}] \leftarrow M\boldsymbol{v}$ is the vector of shares, where $s_i \in \mathbb{Z}_q$ belongs to shareholder $\rho(i)$ for each $i \in [1, \cdots, |\mathbb{A}|]$.*

[1] We can consider $\mathbb{A}$ as an access control tree, as illustrated in Figure **??** and Figure **??**, where $|\mathbb{A}|$ denotes the number of leaf nodes—or equivalently, the number of secret shares. The value of l, however, depends on the specific parsing algorithm introduced in [**?**,**?**].

– $s \leftarrow \mathsf{LSSSRecon}(\mathbb{A}, Q)$*: Let* $Q = \{s_i\}_{I_Q}$ *be an authorized set for* $\mathbb{A}$*, where* $I_Q = \{i|\rho(i) \in Q\}$ *is the row indices in matrix* M *corresponding to* $\mathbb{A}$*. Let* $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$ *be the matrix formed by taking the subset of rows in* M *that are indexed by* I_Q*. Only if* Q *is an authorized set, there exists a vector* $\boldsymbol{w}_Q \in \mathbb{Z}_N^{|I_Q|}$ *such that* $\boldsymbol{w}_Q^\top M_Q = [1, 0, \ldots, 0]$*. (Note that* $\boldsymbol{w}_Q$ *can be calculated using Gaussian elimination algorithm.) Further, the secret* s *can be reconstructed by* $\sum_{i \in I_Q} \boldsymbol{w}_i s_i$*.*

Similar to Shamir SS, LSSS should also satisfy ***correctness*** *and* ***secrecy****.*

3.4 Sigma Protocol and NIZK Proof

Let $\mathcal{R}$ be a relation, and define the corresponding language as $L_{\mathcal{R}} = \{x \mid \exists w : (x, w) \in \mathcal{R}\}$. The elements x and w are typically referred to as the statement and the witness, respectively. A Sigma protocol (P_1, P_2, V) is a three-move protocol between a prover and a verifier [**?**]. Firstly, the prover calculates a commitment $a = P_1(x)$; Secondly, the verifier randomly selects a challenge value $e \leftarrow \{0,1\}^\lambda$, where λ is the security parameter; Lastly, the prover returns the response value $z = P_2(x, a, e)$. If $V(x, a, e, z) = 1$, the verifier is possesses that the prover possesses a witness w of the statement x such that $(x, w) \in \mathcal{R}$.

A non-interactive zero-knowledge (NIZK) proof of knowledge about w can be obtained by applying the Fiat-Shamir heuristic [**?**], with a random oracle. Model the random oracle using a random function, $H : \{0,1\}^* \rightarrow \{0,1\}^\lambda$. Consequently, the NIZK proof is denoted as $(a, e = H(x, a), z)$.

The NIZK proof has the properties of completeness, soundness and zero-knowledge. Completeness means that a valid proof always convinces the verifier; Soundness means that false statements cannot produce valid proofs (except negligibly); Zero-Knowledge means that the proof reveals nothing beyond the statement's truth.

4 Generalized Secret Sharing (GSS)

We first define two types of GSS schemes, which differ based on the shared secret, $s \in \mathbb{Z}_q$ or $S \in \mathbb{G}$.

4.1 Definition

Definition 5 (GSS). *A generalized secret sharing (GSS) scheme [?] allows a dealer to share a secret* $s \in \mathbb{Z}_q$ *using arbitrary monotone access structure. The secret* s *can be recovered only given an authorized set. A GSS scheme is defined by the following two algorithms.*

– $\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\} \leftarrow \mathsf{GSSShare}(s, \mathbb{A})$*: Inputs a value* $s \in \mathbb{Z}_q$ *and an access structure* $\mathbb{A}$*, outputs the secret shares* $\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\}$*, where* $|\mathbb{A}|$ *is the number of leaf nodes under access tree* $\mathbb{A}$*.*

– $s \leftarrow \mathsf{GSSRecon}(\mathbb{A}, Q)$*: Inputs an access structure* $\mathbb{A}$ *and an authorized set* $Q \subseteq \{s_1, s_2, \cdots, s_{|\mathbb{A}|}\}$*, reconstructs the secret* s*.*

Definition 6 (GSS on Group $\mathbb{G}$). *Here we consider how to share secret* $S \in \mathbb{G}$ *and the dealer does not know the value* s *such that* $S = g^s$*. The GSS on group* $\mathbb{G}$ *of order* q *is defined as below:*

– $\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\} \leftarrow \mathsf{GrpGSSShare}(S, \mathbb{A})$*: Inputs a value* $S \in \mathbb{G}$ *and an access structure* $\mathbb{A}$*, outputs the secret shares* $\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\}$*, where* $|\mathbb{A}|$ *is the same as in Definition* **??**.
– $S \leftarrow \mathsf{GrpGSSRecon}(\mathbb{A}, Q)$*: Inputs an access structure* $\mathbb{A}$ *and an authorized set* $Q \subseteq \{S_1, S_2, \cdots, S_{|\mathbb{A}|}\}$*, reconstructs the secret* S*.*

Similar to Shamir SS, GSS and GSS on Group $\mathbb{G}$ should also satisfy **correctness** and **secrecy**.

4.2 Shamir SS-based GSS

Then, we initialize each type (Definition **??** and Definition **??**) using two approaches, i.e., Shamir SS-based and LSSS-based. Technically, the Shamir SS-based primitive requires polynomial evaluation in the secret sharing and Lagrange interpolation in the secret recovery; The LSSS-based primitive needs to handle matrix multiplication both in secret sharing and recovery.

(1) ***GSS based on Shamir SS:*** Benaloh et al.[**?**] had introduced the construction, which can be obtained by replacing GrpShare (and GrpRecon) with Share (and Recon) in Figure **??**. Note that the sharing phase is carried out by traversing the access control structure tree $\mathbb{A}$ from top to bottom, while the recovery process is performed by traversing the tree from bottom to top. We omit the security proofs of **correctness** and **secrecy**, which is proven by Benaloh et al.[**?**].

(2) ***GSS on Group $\mathbb{G}$ based on Shamir SS:*** Without loss of generalization, an access structure $\mathbb{A}$ is a set of shareholders who are constraint by the threshold gate (refer to Figure **??** as an example). The non-leaf nodes are threshold gate values and the leaf nodes are identifiers of shareholders. The GSS scheme attaches a share for each (non-leaf or leaf) node and it is described by Figure **??**.

For any node in GSS scheme on $\mathbb{G}$, the **correctness** and **secrecy** can be derived directly from those of Shamir SS scheme on $\mathbb{G}$, which is given by Lemma **??**. Thus, the **correctness** and **secrecy** of GSS on group $\mathbb{G}$ based on Shamir SS (Figure **??**) are satisfied.

Lemma 1. *Shamir SS on group* $\mathbb{G}$ *(Definition* **??***) satisfies* ***correctness*** *and* ***secrecy****.*

Proof. **Correctness**: The shares are $S_i = S \cdot g^{f(i)}$, $i \in [0, n]$. Following the correctness of Shamir SS scheme, inputs a set of any t shares Q, we can reconstruct

Functionality GSS on Group $\mathbb{G}$ using Shamir SS

$\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\} \leftarrow \mathsf{GrpGSSShare}(S, \mathbb{A})$:

set $F_R \leftarrow S$ for the root node R.

For non-leaf node x with n_x children nodes and threshold-gate t_x:

$\{S_{x_1}, S_{x_2}, \cdots, S_{xn_x}\} \leftarrow \mathsf{GrpShare}(F_x, n_x, t_x)$

set $F_z \leftarrow S_{x_j}$, where z is the jth child of x

For the ith leaf node z in $\mathbb{A}$:

$S_i = S_{x_j}$, where z is the jth child of its parent node x

$(S) \leftarrow \mathsf{GrpGSSRecon}(\mathbb{A}, Q)$:

Denote *path* as the satisfied path (cf. Corollary **??**)

For each node x in *path* from leaf to root:

$F_x \leftarrow \mathsf{GrpRecon}(Q_x)$, where $Q_x \subseteq \{F_{x_1}, \cdots, F_{x_{n_x}}\}$ generated at node x and $|Q_x| \geq t_x$. (**Remark:** if x is a leaf node, $F_x = Q_x$.)

$S \leftarrow F_R$, where R is the root node.

Fig. 3: GSS scheme on Group $\mathbb{G}$ based on Shamir SS

the secret $\prod_{i\in I} S_i^{\prod_{j\in I, j\neq i} \frac{-j}{i-j}} = S \cdot g^{f(0)} = S$, where I is the set containing indexes of the shares Q. **Secrecy**: If less than t shares can reconstruct the secret, then we have $\sum_{i\in I} f(i) \prod_{j\in I, j\neq i} \frac{-j}{i-j} = f(0), |I| < t$, which violates the security properties of Shamir SS scheme.

4.3 LSSS-based GSS

(1) ***GSS based on LSSS:*** It is obvious that LSSS and GSS schemes have the same algorithms, including inputs and ouputs. Figure **??** demonstrate that LSSS is an instance of GSS. Therefore, the **correctness** and **secrecy** are inherently guaranteed.

Functionality GSS using LSSS

$\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\} \leftarrow \mathsf{GSSShare}(s, \mathbb{A})$:

$\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\} \leftarrow \mathsf{LSSSShare}(s, \mathbb{A})$

$(s) \leftarrow \mathsf{GSSRecon}(\mathbb{A}, Q)$:

$s \leftarrow \mathsf{LSSSRecon}(\mathbb{A}, Q)$

Fig. 4: GSS based on LSSS

(2) ***GSS on Group $\mathbb{G}$ using LSSS:*** We first depict how to construct LSSS on group $\mathbb{G}$ by Figure **??**. The construction resembles the LSSSShare and

LSSSRecon algorithms of the LSSS scheme, as defined in Definition **??**. We prove that it satisfies the **correctness** and **secrecy** by Lemma **??**.

Functionality LSSS on Group $\mathbb{G}$

$\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\} \leftarrow \mathsf{GrpLSSSShare}(S, \mathbb{A})$: denote $S = g^s$ and s is unknown.

parse $\mathbb{A}$ as (M, ρ) leveraging the algorithms in [**?**] or [**?**]

$\boldsymbol{v} = [1, v_2, \ldots, v_l]^\top$, where $v_2, \ldots, v_l \xleftarrow{R} \mathbb{Z}_q$.

$(s_1, \cdots, s_{|\mathbb{A}|}) \leftarrow M\boldsymbol{v}$

$S_i = S^{s_i} = g^{s \cdot s_i}$ for $i \in [1, \cdots, |\mathbb{A}|]$

$(S) \leftarrow \mathsf{GrpLSSSRecon}(\mathbb{A}, Q)$:

parse $\mathbb{A}$ as (M, ρ)

calculate I_Q as $I_Q = \{i | \rho(i) \in Q\}$

set $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$, where M_Q is the subset of rows in M that are indexed by I_Q

find $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \ldots, 0]$

$S \leftarrow \prod_{i \in I_Q} S_i^{w_i}$

Fig. 5: LSSS on Group $\mathbb{G}$

Lemma 2. *LSSS on group $\mathbb{G}$ (Figure **??**) satisfies **correctness** and **secrecy**.*

Proof. **Correctness**: The shares are $S_i = S^{s_i} = g^{s \cdot s_i}$ for $i \in [1, \cdots, |\mathbb{A}|]$. Q is a authorized set with $I_Q = \{i | \rho(i) \in Q\}$ and $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$, which means there exists $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \ldots, 0]$. Following the correctness of LSSS scheme, we have $\sum_{i \in I_Q} w_i \cdot s_i = 1$, thus $S = \prod_{i \in I_Q} S_i^{w_i} = \prod_{i \in I_Q} g^{s \cdot s_i \cdot w_i} = (g^s)^{\sum_{i \in I_Q} w_i \cdot s_i} = g^s$. **Secrecy**: If unauthorized set Q can reconstruct $g^s = \prod_{i \in I_Q} S_i^{w_i}$, then it can find $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \ldots, 0]$, which violates the security properties of LSSS scheme, as only authorized sets can find such w_Q.

Then, we apply the construction of Figure **??** to construct GSS on group $\mathbb{G}$, as Figure **??** presents. It can be observed that the algorithm GrpGSSShare (w.r.t. GrpGSSRecon directly executes GrpLSSSShare (w.r.t. GrpLSSSRecon). Apparently, LSSS on group $\mathbb{G}$ can be regarded as an instance of GSS on group $\mathbb{G}$. Thus, **correctness** and **secrecy** are inherently guaranteed by Lemma **??**.

4.4 GSS Testing

Note that GSSRecon works well given any authorized set Q and the corresponding access structure $\mathbb{A}$ as input. In practice, $Q^* \subseteq Q$ is the minimal

Functionality GSS on Group $\mathbb{G}$ using LSSS

$\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\} \leftarrow \mathsf{GrpGSSShare}(S, \mathbb{A})$:

$\{S_1, S_2, \cdots, S_{|\mathbb{A}|}\} \leftarrow \mathsf{GrpLSSSShare}(S, \mathbb{A})$

$(S) \leftarrow \mathsf{GrpGSSRecon}(\mathbb{A}, Q)$:

$S \leftarrow \mathsf{GrpLSSSRecon}(\mathbb{A}, Q)$

Fig. 6: GSS on Group $\mathbb{G}$ based on LSSS

authorized set in reconstructing s and the shares in $Q - Q^*$ are not used. In case given a superset $Q \supset Q^*$, a user may recover contradicting secrets if Q is provided by malicious shareholders or dealer. How to efficiently design the algorithm is non-trivial, since arbitrary access structures may exist in applications. Therefore, we need an efficient algorithm to support validity check given any authorized set Q. We name the process as GSS testing.

Definition 7 (GSS Testing). *Given an access structure $\mathbb{A}$ and shares set Q, GSS testing refers to the process of determining whether all elements in Q are valid shares generated under $\mathbb{A}$.*

Here, we present two approaches to implementing GSS testing for a set Q with $|Q| = |\mathbb{A}|$. These two approaches are applicable for both the Shamir SS-based and LSSS-based GSS.

① Randomly select a minimal authorized set $Q^* \subseteq Q$ together with the access structure $\mathbb{A}$ as input. Using the Lagrange interpolation algorithm, reconstruct the polynomials for each non-leaf node of the access tree $\mathbb{A}$ from the bottom up. Then, recompute the shares at the evaluation points and compare them against those in $Q - Q^*$ to verify share equivalence. This approach reliably determines whether the shares in Q are correctly generated, and it introduces no soundness error.

② As is known, Shamir SS shares are equivalent to Reed-Solomon error correcting code and dual codes can be applied to verify correctness of the shares with soundness error $\frac{1}{q}$ [**?**]. Therefore, for the access tree $\mathbb{A}$, we can recursively reconstruct a secret value (using Lagrange interpolation) at each non-leaf node from the bottom up, while simultaneously verifying the correctness of its child-node secret values (or shares) via the dual code. Evidently, the soundness error of this approach is bounded above by $\frac{1}{q}$ for any access structure $\mathbb{A}$.

Further, we define $\mathsf{GSSReconWithVrf}$, an extension of $\mathsf{GSSRecon}$, which not only recovers the secret but also performs GSS testing.

Functionality GSS Reconstruction with testing

$(0/1) \leftarrow \mathsf{GSSReconWithVrf}(\mathbb{A}, Q, s)$:

find a minimum satisfied set $Q^* \subseteq Q$

$s' \leftarrow \mathsf{GSSRecon}(\mathbb{A}, Q^*)$

$s \stackrel{?}{=} s'$ & GSS Testing

5 Publicy Verifiable Generalized Secret Sharing (PVGSS)

5.1 Definition

Definition 8 (PVGSS). *A Publicly verifiable generalized secret sharing (PVGSS) scheme allows a dealer to split a secret using an arbitrary monotone access structure in a publicly verifiable manner. Hence, shares are encrypted and can be delivered in a public communication channel. Besides, the correctness of the encrypted shares can be verified by any external party. The PVGSS scheme includes five algorithms.*

- $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{PVGSSSetup}(\lambda)$*: Inputs security parameter* λ*, outputs a key pair* $(\mathsf{sk}_i, \mathsf{pk}_i)$ *for each shareholder* P_i*.*
- $(\{C_i\}, \mathsf{prf}_s) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}, \{\mathsf{pk}_i\})$*: Inputs an access structure* $\mathbb{A}$ *and the public keys* $\{\mathsf{pk}_i\}$*, outputs the encrypted shares* $\{C_i\}$ *and the corresponding NIZK proofs* prf_s *for a secret* s *randomly chosen from* $\mathbb{Z}_q$*.*
- $(0/1) \leftarrow \mathsf{PVGSSVerify}(\{C_i\}, \mathsf{prf}_s, \mathbb{A})$*: Inputs encrypted shares* $\{C_i\}$*, NIZK proofs* prf_s *and the access structure* $\mathbb{A}$*, outputs 1 if the shares are correctly encrypted. Otherwise, outputs 0.*
- $(S_i, \mathsf{prf}_{S_i}) \leftarrow \mathsf{PVGSSPreRecon}(C_i, \mathsf{sk}_i)$*: Inputs ciphertext* C_i *and the corresponding secret key* sk_i*, outputs a decrypted share* S_i *and the attached NIZK proof* prf_{S_i}*.*
- $(0/1) \leftarrow \mathsf{PVGSSKeyVrf}(C_i, S_i, \mathsf{prf}_{S_i}, \mathsf{pk}_i)$*: Inputs the encrypted share* C_i*, the decrypted share* S_i*, the NIZK proof* prf_{S_i} *and the public key* pk_i*, outputs 1 if* S_i *is corrected decrypted from* C_i*.*
- $(S) \leftarrow \mathsf{PVGSSRecon}(\mathbb{A}, Q)$*: Inputs the access structure* $\mathbb{A}$ *and a set of decrypted shares* Q*, outputs the recovered secret* $S(= g^s)$ *given* Q *is authorized.*

PVGSS scheme is required to achieve below security properties, which are formally defined in Appendix **??**.

• **Correctness**: The scheme ensures that if the secret sharing process is correctly followed, any authorized subset of participants can reconstruct the secret S.

• **Verifiability**: Anyone can verify that the encrypted shares provided by the dealer are correct without knowing the actual secret. Besides, users can also check correctness of the decrypted shares in the reconstruction phase.

• **IND1-secrecy**: The secret remains hidden from unauthorized parties prior to the reconstruction phase.

5.2 PVGSS Construction Based on GSS

Usually, PVSS schemes are constructed based on SS. Informally speaking, PVSS combines SS and public key encryption with verifiability. Intuitively, PVGSS can be constructed similarly by replacing SS with GSS. We leverage the original GSS scheme in the sharing phase and employ GSS on group $\mathbb{G}$ in the reconstruction phase. Figure **??** illustrate our mindset of constructions vividly. Figure **??** demonstrates the general PVGSS construction by taking ad-

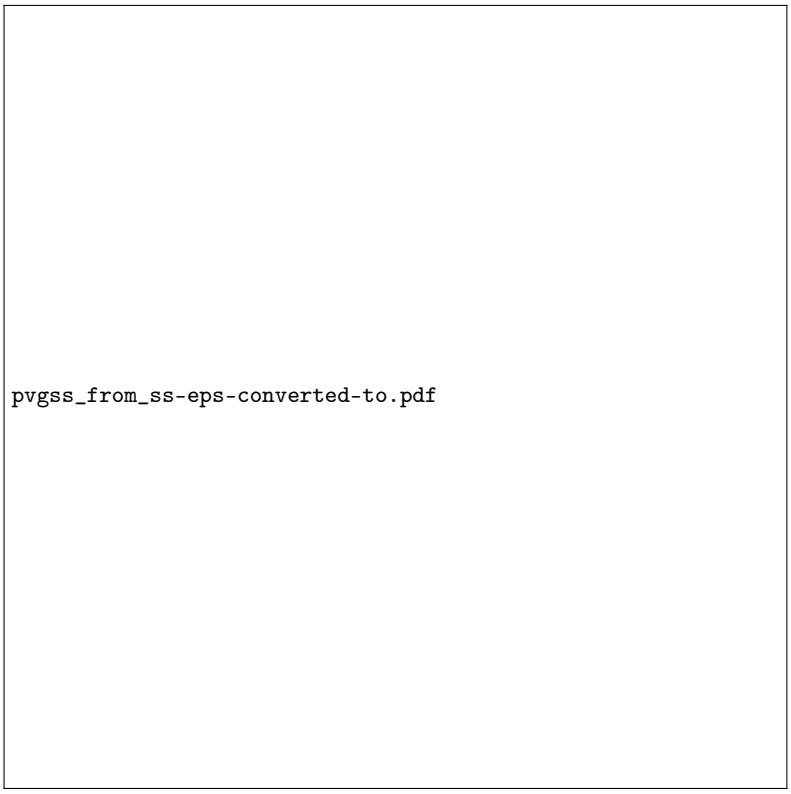

Fig. 7: High-level overview of our PVGSS constructions

vantage of GSS scheme and NIZK proofs. The NIZK proofs are acquired by incorporating the Sigma protocol [**?**] and the Fiat-Shamir heuristic [**?**]. The initiation phase comprises the PVGSSSetup algorithm. The sharing phase involves the PVGSSShare and PVGSSVerify algorithms. The reconstruction phase includes PVGSSPreRecon, PVGSSKeyVrf, and PVGSSRecon algorithms.

In the PVGSSSetup algorithm, each shareholder j generates its key pair $(\mathsf{sk}_i, \mathsf{pk}_i)$, where $\mathsf{pk}_i = g^{\mathsf{sk}_i}$ and sk_i is randomly chosen from $\mathbb{Z}_q$.

In the PVGSSShare algorithm, the dealer first invokes GSSShare algorithm for secret $s \in \mathbb{Z}_q$ to generate secret shares based on the access structure $\mathbb{A}$. Note that a PVGSS is not merely a combination of multiple PVSS instances, since it outputs encrypted shares only for the leaf nodes. As previously introduced, $|\mathbb{A}|$ denotes the number of leaf nodes in the access structure $\mathbb{A}$. Then, it hides the $|\mathbb{A}|$ shares directly using the targeted shareholder's public key, i.e., pk^{s_i}. Next, it executes the Sigma protocol and Fiat-Shamir heuristic to generate NIZK proofs (cf. Section **??**). Specifically, $\{C_i'\}$, c, $\{\hat{s}_i\}$ are the commitment, the challenge value and the response value, respectively. Therefore, the NIZK proof prf_s is $(\{C_i'\}, c, \{\hat{s}_i\})$. To this end, the dealer has successfully generated encrypted se-

cret shares $\{C_i\}$ and the corresponding NIZK proof prf_s which can be adopted to prove the dealer's honesty. Different from the LDEI protocol in Albatross [**?**], which directly outputs the polynomial $\hat{p}(X)$ as part of the Sigma protocol response values, PVGSS transmits $|\mathbb{A}|$ polynomial evaluation points $\{\hat{s}_i\}$ of $\hat{p}(X)$.

Functionality PVGSS Based on GSS

$\underline{(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{PVGSSSetup}(\lambda) :}$

$\mathsf{sk}_i \xleftarrow{R} \mathbb{Z}_q$

$\mathsf{pk}_i \leftarrow g^{\mathsf{sk}_i}$

$\underline{(\{C_i\}, \mathsf{prf}_s) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}, \{\mathsf{pk}_i\}) :}$

$s \xleftarrow{R} \mathbb{Z}_q$

$\{s_1, s_2, \cdots, s_{|\mathbb{A}|}\} \leftarrow \mathsf{GSSShare}(s, \mathbb{A})$

$C_i = \mathsf{pk}_i^{s_i} \ \forall i \in [1, ..., |\mathbb{A}|]$

$s' \xleftarrow{R} \mathbb{Z}_q$

$(s'_1, ..., s'_{|\mathbb{A}|}) = \mathsf{GSSShare}(s', \mathbb{A})$

$C'_i = \mathsf{pk}_i^{s'_i} \ \forall i \in [1, ..., |\mathbb{A}|]$

$c \leftarrow H(\{C_i\}, \{C'_i\})$

$\hat{s} \leftarrow s' - c \cdot s$

$\hat{s}_i \leftarrow s'_i - c \cdot s_i \ \forall i \in [1, ..., |\mathbb{A}|]$

$\mathsf{prf}_s \leftarrow (\{C'_i\}, c, \hat{s}, \{\hat{s}_i\})$

$\underline{(0/1) \leftarrow \mathsf{PVGSSVerify}(\{C_i\}, \mathsf{prf}_s, \mathbb{A}) :}$

$C'_i \stackrel{?}{=} C_i^c \cdot \mathsf{pk}_i^{\hat{s}_i} \ \forall i \in [1, ..., |\mathbb{A}|]$

$\hat{s} \stackrel{?}{=} \mathsf{GSSReconWithVrf}(\mathbb{A}, \{\hat{s}_i\}_{i \in [1, |\mathbb{A}|]})$.

$\underline{(S_i, \mathsf{prf}_{S_i}) \leftarrow \mathsf{PVGSSPreRecon}(C_i, \mathsf{sk}_i) :}$

$S_i = C_i^{1/\mathsf{sk}_i} = g^{s_i}$

$\mathsf{prf}_{S_i} = DLEQ(S_i, C_i, g, \mathsf{pk}_i, \mathsf{sk}_i)$

$\underline{(0/1) \leftarrow \mathsf{PVGSSKeyVrf}(C_i, S_i, \mathsf{prf}_{S_i}, \mathsf{pk}_i) :}$

$\mathsf{true} \stackrel{?}{=} DLEQVrf(S_i, C_i, g, \mathsf{pk}_i, \mathsf{prf}_{S_i})$

$\underline{(S) \leftarrow \mathsf{PVGSSRecon}(\mathbb{A}, Q) :}$

$S \leftarrow \mathsf{GrpGSSRecon}(\mathbb{A}, Q)$ where $S = g^s$ and $Q \subseteq \{S_1, \cdots, S_{|\mathbb{A}|}\}$ is an authorized set of $\mathbb{A}$.

Fig. 8: PVGSS scheme based on GSS and NIZK proofs

In the $\mathsf{PVGSSVerify}$ algorithm, any (external) party can check whether the dealer has honestly performed the sharing phase phase. Due to the presence of $|\mathbb{A}|$ independent shares $\{s_i\}$, the verifier needs to perform verification using two

equations. The first equation (executed $|\mathbb{A}|$ times) is equivalent to the Schnorr signature [**?**], enabling the dealer to prove knowledge of each individual share s_i. The second equation implies that all the response values $\{\hat{s}_i\}$ are correctly generated using the challenge value c. The second equation allows the dealer to prove that the shares embedded in $\{C_i\}$ are bound to the secret s and $S = g^s$. Refer to Theorem **??** for details about the correctness.

In the PVGSSPreRecon algorithm, the shareholder, holding sk_i, decrypts C_i with its secret key sk_i and obtains $S_i = g^{s_i}$. Additionally, the shareholder employs the discrete logarithm equality (DLEQ) algorithm [**?**] to generate a non-interactive zero-knowledge (NIZK) proof prf_{S_i}, demonstrating that C_i and pk_i share the same exponent sk_i.

The PVGSSKeyVrf algorithm directly invokes the corresponding DLEQ verification algorithm, *DLEQVrf*, to verify the correctness of S_i provided by the shareholder. Note that no information about sk_i is leaked during this process.

Given enough authorized decrypted shares, i.e., an authorized set Q, the PVGSSRecon algorithm can reconstruct a secret S using GrpGSSRecon algorithm. Note that GrpGSSRecon, rather than GSSRecon, is used to recover the secret, since in our PVGSS scheme the secret S resides in $\mathbb{G}$, consistent with many prior PVSS constructions [**?**,**?**].

5.3 Complexity Analysis

Computational Complexity: The computational costs of matrix operations (including generation and multiplication in the LSSS-based construction) and polynomial operations (including evaluation and Lagrange interpolation in the Shamir SS-based construction), are negligible compared to group exponentiation in $\mathbb{G}$. As such, these operations are excluded from the complexity analysis.

In the sharing phase, the PVGSSShare algorithm costs $2|\mathbb{A}|$ group exponentiations, where each C_i and C_i' requires an exponentiation. The PVGSSVerify algorithm requires $2|\mathbb{A}|$ exponentiations to verify the correctness of each C_i. Therefore, the distribution complexity is $O(|\mathbb{A}|)$. In the reconstruction phase, the PVGSSPreRecon algorithm costs 3 exponentiations for each shareholder, including the calculation of prf_{S_i} from DLEQ. The PVGSSKeyVrf algorithm requires 4 exponentiations as required by *DLEQVrf*. The PVGSSRecon algorithm takes $|Q|$ exponentiations, where Q denotes the authorized set for the access structure $\mathbb{A}$. Thus, the reconstruction complexity is $O(|Q|)$.

Communicational Complexity: In the PVGSS sharing phase, the dealer outputs $(\{C_i\}, \mathsf{prf}_s)$, which consists of $2|\mathbb{A}|$ $\mathbb{G}$ elements and $(|\mathbb{A}|+2)$ $\mathbb{Z}_q$ elements. During the reconstruction phase, each user needs to collect $(3\mathbb{G} + 2\mathbb{Z}_q)|Q|$ elements, as each correct prf_{S_i} contains 2 elements on $\mathbb{G}$ and 2 elements on $\mathbb{Z}_q$. Therefore, the communication complexity is $O(|\mathbb{A}|)$ during the sharing phase and $O(|Q|)$ during the reconstruction phase, respectively.

5.4 Security Analysis

Theorem 1 (Correctness). *The PVGSS construction satisfies correctness defined by Definition* **??**.

Proof. If the dealer is honest and outputs $\{C_i\}$ and prf_s using $\mathsf{GSSShare}$ based on the access structure $\mathbb{A}$, then $\mathsf{PVGSSVerify}(\{C_i\}, \mathsf{prf}_s, \mathbb{A}) = 1$. That is implied by Schnorr signature [**?**], where each $(C_i, c, (C_i', \hat{s_i}))$ contained in prf_s comprise of a Sigma protocol [**?**]. Further, the $\mathsf{GSSReconWithVrf}$ algorithm can successfully recover $\hat{s}$ due to the additive homomorphism of Shamir SS and LSSS schemes. If $\mathsf{PVGSSPreRecon}$ is applied to (C_i, sk_i) honestly, then $S_i = g^{s_i}$ and $\mathsf{PVGSSKeyVrf}(C_i, S_i, \mathsf{prf}_{S_i}, \mathsf{pk}_i) = 1, \forall i \in I_Q$. Given an authorized set Q of $\mathbb{A}$, we have the reconstructed value $S' \leftarrow \mathsf{GrpGSSRecon}(\mathbb{A}, Q)$ which in turn equals $S = g^s$, due to **correctness** of GSS on group $\mathbb{G}$. The correctness has been proved in Section **??** and Section **??**. Therefore, the PVGSS satisfies correctness defined by Definition **??**.

Theorem 2 (Verifiability of Encrypted Shares). *If NIZK proof with soundness error negligible in λ, then PVGSS has verifiability of encrypted shares, defined by Definition* **??**.

Proof. If $\mathsf{PVGSSVerify}(\{C_i\}, \mathsf{prf}_s, \mathbb{A}) = 1$, then except with NIZK soundness error $negl(\lambda)$, there exists s_i such that $C_i = \mathsf{pk}_i^{s_i}$ and $C_i' = C_i^c \cdot \mathsf{pk}_i^{\hat{s_i}}$. Further, $\hat{s} = \mathsf{GSSReconWithVrf}(\mathbb{A}, \{\hat{s}_i\})$ holds. If $\{C_i\}$ are not valid shares of a PVGSS secret, then $\mathsf{GSSReconWithVrf}(\mathbb{A}, \{s_i\})$ is denoted as $\perp$. Further, $\mathsf{GSSReconWithVrf}(s_i') = \mathsf{GSSReconWithVrf}(\mathbb{A}, \{\hat{s_i} + c \cdot s_i\}) = \mathsf{GSSReconWithVrf}(\mathbb{A}, \{\hat{s_i}\}) + \mathsf{GSSReconWithVrf}(\mathbb{A}, \{c \cdot s_i\}) = \hat{s} + c \cdot \perp$. It means that $\{s_i'\}$ are not valid shares of a secret. Therefore, no valid s' and s exists to generate $\hat{s}$, violating $\hat{s}$ as response value of Sigma protocol with NIZK soundness error $negl(\lambda)$. Therefore, the PVGSS satisfies verifiability of encrypted shares by Definition **??**.

Theorem 3 (Verifiability of Share Decryption). *If a shareholder communicates a decryption share $\hat{S}_i$ in the reconstruction phase, then it is detected with probability 1, defined by Definition* **??**.

Proof. The verifiability of share decryption is evident from the properties of the DLEQ algorithm [**?**], where sk_i serves as the common exponent in both C_i and pk_i.

Theorem 4 (IND1-secrecy). *The PVGSS is IND1-secrecy, defined by game Definition* **??**, *under DDH assumption.*

Proof. Firstly, the NIZK proofs prf_s of $\mathsf{PVGSSShare}$ reveal nothing about the secret s or shares $\{s_i\}$, due to zero-knowledge property. Then, we argue that, if there exists an adversary $\mathcal{A}_{IND}$ which can break the **IND1-secrecy** property of PVGSS, then there exists an adversary $\mathcal{A}_{DDH}$ which can use $\mathcal{A}_{IND}$ to break DDH assumption.

Fig. 9: A maximum unauthorized set is shown in green, while an unsatisfied path is highlighted in orange.

Shamir SS-based PVGSS: The security game requires $\mathcal{A}_{IND}$ controls a unauthorized set. Thus, there exists a path in which all nodes are not satisfied by the unauthorized set, as stated in Corollary **??**. Taking Figure **??** as an example. $\mathcal{A}_{IND}$ can query a maximum set of decrypted shares for nodes: $\{P_{1,*}, P_{2,*}, P_{3,2}\}$.[2] Thus, nodes $(N_0, N_2, P_{3,1})$ compose an unsatisfied path. From leaf to root in the unsatisfied path, we assume each node is not satisfied by a maximum unauthorized set. That means, we only need to consider a (t, n) threshold access structure $\mathbb{A}^*$, and assume that $\mathcal{A}_{IND}$ corrupts the $t-1$ first shareholders.

Let $(h, h^\alpha, h^\beta, h^\gamma)$ be an instance of the DDH problem. Then, $\mathcal{A}_{DDH}$ using $\mathcal{A}_{IND}$ can simulate an security game as follows:

1. $\mathcal{A}_{IND}$ generates a (t, n) threshold access structure $\mathbb{A}^*$ and sends it to the challenger. $\mathcal{A}_{IND}$ sets the maximum unauthorized set index $U = [1, \cdots, t-1]$. $\mathcal{A}_{IND}$ sends $(\mathbb{A}^*, U)$ to the $\mathcal{A}_{IND}$.
2. The challenger sets $g = h^\alpha$. For $t \leq i \leq n$, $\mathcal{A}_{DDH}$ randomly choose $r_i \xleftarrow{R} \mathbb{Z}_q$ and sends the values $\mathsf{pk}_i = h^{r_i}$ to $\mathcal{A}_{IND}$. This implicitly defines $\mathsf{sk}_i = r_i/\alpha$ for $i \in [t, n]$, though α is unknown to the challenger.
3. For $1 \leq i \leq t-1$, $\mathcal{A}_{IND}$ randomly chooses sk_i and sets $\mathsf{pk}_i = g^{\mathsf{sk}_i}$, then, it sends these to the challenger.

[2] The maximum authorized set is not unique.

4. For $1 \leq i \leq t-1$, the challenger samples $s_i \xleftarrow{R} \mathbb{Z}_q$ and sets $C_i^* = \mathsf{pk}_i^{s_i}$.
   For $1 \leq i \leq t-1$, the challenger randomly chooses s_i from $\mathbb{Z}_q$ and calculates h^{s_i}. Then, given t points, i.e., $\{h^{s_i}\}$ and h^β, $\mathcal{A}_{DDH}$ can use Lagrange interpolation to compute $S_i^* = h^{s_i} = h^{p(i)}$ for $t \leq i \leq n$, where $p(\cdot)$ is the interpolated polynomial with degree $t-1$ such that $h^{p(0)} = h^\beta$. Further, it also generates shares $C_i^* = (S_i^*)^{r_i} = \mathsf{pk}_i{}^{s_i}$.
   To simulate the NIZK proof prf_β, $\mathcal{A}_{DDH}$ randomly chooses a $(t-1)$-degree polynomial $\hat{p}(x)$ and $c \in \mathbb{Z}_q$. Then, $\mathcal{A}_{DDH}$ calculates $\hat{s}_i = \hat{p}(i)$ and sets $C_i'^* = (C_i^*)^c \cdot \mathsf{pk}_i^{\hat{s}_i}$ for $1 \leq i \leq n$. Note that the random oracle $H(\cdot)$ is programmed so that $c \equiv H(\{C_i^*\}, \{C_i'^*\})$. Apparently, the simulated tuple $(\{C_i'^*\}, c, \hat{p}(0), \{\hat{s}_i\})$ forms a valid NIZK proof prf_β for the sharing phase.
   Finally, it sends $\{C_i^*\}$, prf_β and h^γ to $\mathcal{A}_{IND}$, where h^γ plays the role of x_b in the IND1-secrecy game.
5. Output: $\mathcal{A}_{IND}$ make a guess b'.
   If $b' = 0$, $\mathcal{A}_{DDH}$ guess that $\gamma = \alpha \cdot \beta$, if $b' = 1$, $\mathcal{A}_{DDH}$ guess that γ is a random element. The information that $\mathcal{A}_{IND}$ receives in step 4). is distributed exactly like a sharing of the value $g^\beta = h^{\alpha \cdot \beta}$. If h^γ sent to $\mathcal{A}_{IND}$ is the secret shared by PVGSS, $\gamma = \alpha \cdot \beta$. So the advantage of $\mathcal{A}_{DDH}$ is the same as the advantage of $\mathcal{A}_{IND}$.

LSSS-based PVGSS: Consider $\mathbb{A}^*$ is a LSSS access structure (M^*, ρ), each row corresponds to one shareholder. Assume $\mathcal{A}_{IND}$ can corrupts a max unauthorized set U of $\mathbb{A}^*$. Let $I_U = \{i | \rho(i) \in U\}$ be the rows by the row indexes U. To maximize the attacking capacity of $\mathcal{A}_{IND}$, the set I_U may include partial rows for some party $\rho(i)$. Taking Figure **??** as an example, the row corresponding to leaf node $P_{3,2}$ is included in I_U, while the row corresponding to $P_{3,1}$ is not. M_i denotes the ith row of M and $M_{i,j}$ denotes the ith row and jth column of M.

Let $(h, h^\alpha, h^\beta, h^\gamma)$ be an instance of the DDH problem. Then $\mathcal{A}_{DDH}$ using $\mathcal{A}_{IND}$ can simulate an IND game as follows:

1. $\mathcal{A}_{IND}$ generates an generalized access structure $\mathbb{A}^* = (M^*, \rho)$ and sends (M^*, ρ, I_U) to the challenger.
2. The challenger sets $g = h^\alpha$. For $i \notin I_U$, $\mathcal{A}_{DDH}$ randomly chooses $r_i \xleftarrow{R} \mathbb{Z}_q$ and sends the values $\mathsf{pk}_i = h^{r_i}$ to $\mathcal{A}_{IND}$. This implicitly defines $\mathsf{sk}_i = r_i/\alpha$ for $i \notin I_U$, though α is unknown to the challenger.
3. For $i \in I_U$, $\mathcal{A}_{IND}$ randomly chooses sk_i and sets $\mathsf{pk}_i = g^{\mathsf{sk}_i}$, then, it sends these to the challenger.
4. Given M^*, there exists a vector $\boldsymbol{w} = [w_1, \ldots, w_{|\mathbb{A}^*|}] \in \mathbb{Z}_N^{|\mathbb{A}^*|}$ such that $\boldsymbol{w}^\top M^* = [1, 0, \ldots, 0]$. For $j \in I_U$, the challenger samples $s_j \xleftarrow{R} \mathbb{Z}_q$ and sets $C_j^* = \mathsf{pk}_j^{s_j}$. Since I_U is a max unauthorized set index of $\mathbb{A}^*$, given any additional row $i \notin I_U$, the access structure will be satisfied and the secret h^β can be recovered. Therefore, $\mathcal{A}_{DDH}$ computes $S_i^* = h^{s_i} = (\frac{h^\beta}{h^{\sum_{j \in I_U} w_j s_j}})^{\frac{1}{w_i}}$ for all $i \notin I_U$ and then it generates shares $C_i^* = (S_i^*)^{r_i} = \mathsf{pk}_i{}^{s_i}$.
   To simulate the NIZK proof prf_β, $\mathcal{A}_{DDH}$ randomly chooses $\hat{s} \in \mathbb{Z}_q$ and $c \in \mathbb{Z}_q$. Then, $\mathcal{A}_{DDH}$ computes shares $\{\hat{s}_i\}$ for secret $\hat{s}$ under the access structure

$\mathbb{A}^*$ and sets $C_i'^* = (C_i^*)^c \cdot \mathsf{pk}_i^{\hat{s}_i}$ for all i. Note that the random oracle $H(\cdot)$ is programmed so that $c \equiv H(\{C_i^*\}, \{C_i'^*\})$. Apparently, the simulated tuple $(\{C_i'^*\}, c, \hat{s}, \{\hat{s}_i\})$ forms a valid NIZK proof prf_β for the sharing phase.
Finally, the challenger sends $(\{C_i^*\}, \mathsf{prf}_\beta)$ and h^γ to $\mathcal{A}_{IND}$, where h^γ plays the role of x_b in the ND1-secrecy game.

5. Output: $\mathcal{A}_{IND}$ make a guess b'.
If $b' = 0$, $\mathcal{A}_{DDH}$ guess that $\gamma = \alpha \cdot \beta$, if $b' = 1$, $\mathcal{A}_{DDH}$ guess that γ is a random element. The information that $\mathcal{A}_{IND}$ receives in step 4). is distributed exactly like a sharing of the value $g^\beta = h^{\alpha \cdot \beta}$. If h^γ sent to $\mathcal{A}_{IND}$ is the secret shared by PVGSS, $\gamma = \alpha \cdot \beta$. So the advantage of $\mathcal{A}_{DDH}$ is the same as the advantage of $\mathcal{A}_{IND}$.

6 DEX Based on PVGSS

Centralized exchanges (CEXs), such as Binance[3] and Coinbase[4], provide liquidity and usability but rely on central authorities, exposing users to breaches, mismanagement, or censorship and creating single points of failure. Decentralized exchanges (DEXs) emerged to address these issues, offering trustless trading where assets remain user-controlled and transactions secured by blockchain. DEXs inherently tackle the fair exchange problem, proven impossible without a trusted third party [**?**]. Hash time-locked contracts (HTLC)[5] and adaptor signatures [**?**,**?**] enable atomic swaps [**?**,**?**], essential for DEXs. Yet, they lack efficient arbitration, causing delays or risks when parties refuse cooperation, e.g., withholding preimages or signatures. These methods emphasize fairness between traders but neglect broader incentives for liquidity providers (LPs) and arbitrageurs crucial to market growth [**?**]. Automated market maker (AMM) DEXs, such as Uniswap[6], use liquidity pools and algorithms to set prices [**?**], simplifying trading and avoiding strict time-locks. However, LPs face impermanent loss when pool prices shift, while traders encounter slippage in large trades. AMMs also suffer from front-running and miner extractable value (MEV) attacks [**?**,**?**], plus capital inefficiency requiring large liquidity to reduce slippage but yielding suboptimal returns [**?**]. Table **??** compares the different exchanges.

6.1 High-Level Overview

Based on the proposed PVGSS scheme, we design a DEX to allow exchangers to swap tokens fairly and simultaneously. In this research, we merely focus on ERC-20[7] token exchange. The DEX involves two roles: exchangers and watchers. Anyone can be exchangers and watchers. Each exchanger holds specific types of ERC-20 tokens, while multiple watchers collectively form a passive notary

[3] `https://www.binance.com/en`
[4] `https://www.coinbase.com`
[5] `https://en.bitcoin.it/wiki/Hash_Time_Locked_Contracts`
[6] `https://app.uniswap.org`
[7] `https://eips.ethereum.org/EIPS/eip-20`

Table 2: Comparisons of different exchanges

Exchanges	Fariness provider	Arbitrageurs	No SPOF	Arbitration	No MEV/ slippage	Divergence Losslessness
CEX	CEX	N/A	✗	✗	✓	✓
HTLC-based	Miners	N/A	✓	✗	✓	✓
AMM-based	Smart contract	LPs	✓	✗	✗	✗
Ours	Smart contract	Watchers	✓	✓	✓	✓

committee to address potential disputes. Take two exchangers, i.e., Alice and Bob, and n watchers $(W_1, \cdots, W_n)$ as an example. Figure **??** illustrates the concrete access structure incorporating threshold gates. Based on Corollary **??**, we can infer that the set $[Alice, Bob]$ is an authorized set. Apparently, if both Alice and Bob are honest, they can recover any secrets of PVGSS instance. If either exchanger is malicious, say Bob, Alice can recover the secret with help of at least t watchers. Note that t can be set to 1 without affecting fairness.

Fig. 10: The access structure leveraged by Alice and Bob, i.e., $\mathbb{A} = (2\ of\ (Alice, Bob, (t\ of\ (W_1, W_2, \cdots, W_n))))$.

The DEX operates under a commit-and-reveal paradigm. As shown by Figure **??**, the DEX optimistically runs in two communication rounds for Alice

and Bob. In the first round, swap_1, each exchanger commits to a secret using PVGSSShare, where all the $n + 2$ entities are considered as shareholders. The correctness of the commitment is guaranteed by the PVGSSVerify algorithm. In the second round, swap_2, each exchanger reveals its decrypted share using PVGSSPreRecon. The correctness of share decryption is ensured by PVGSSKeyVrf. Then, both Alice and Bob jointly recover each other's secrets using PVGSSRecon. In a pessimistic occasion where a player complains to the passive watchers, who will be involved to resolve dispute using PVGSSPreRecon.

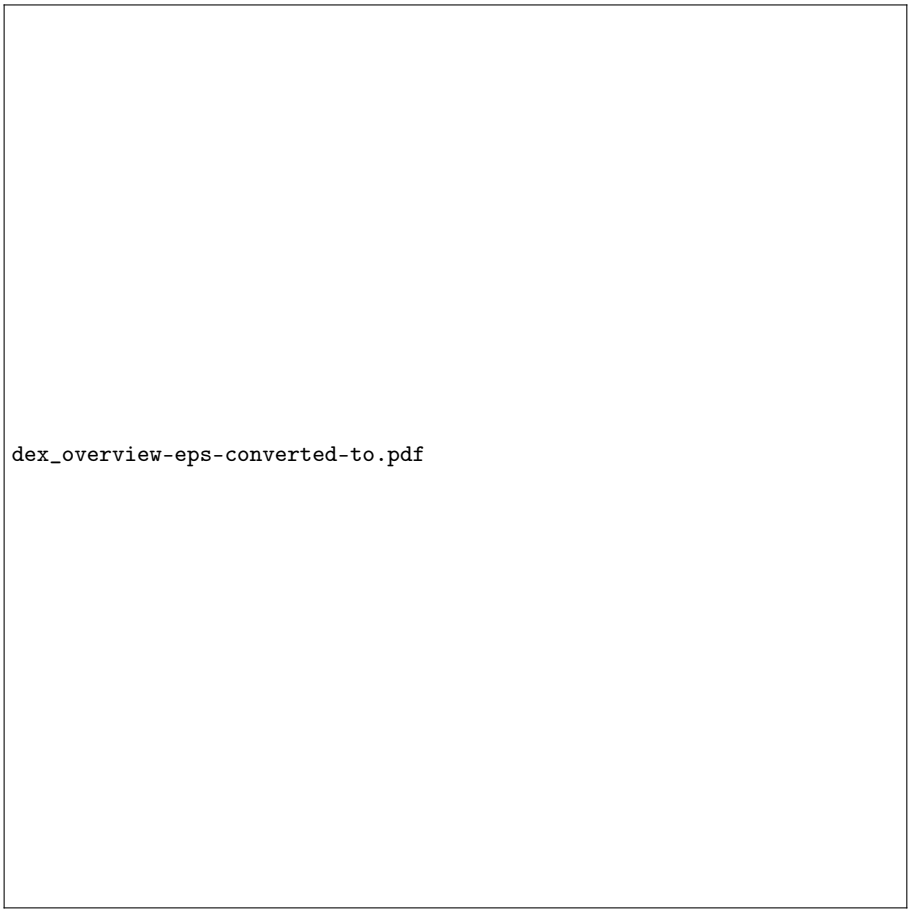

Fig. 11: Overview of the DEX between two exchangers (i.e., Alice and Bob) with passive watchers

6.2 Concrete Design

Ideal functionality. Abstracting away from implementation details, any reasonable decentralized exchange must satisfy three semantic requirements: (i) atomic execution of asset transfers, ensuring that either all swaps complete or none do; (ii) absence of unilateral advantage, meaning that no party can obtain the counterparty's asset without releasing its own; and (iii) public verifiability of outcomes, allowing any observer to determine whether a swap has succeeded or failed. We therefore define the ideal functionality $\mathcal{F}_{\mathrm{DEX}}$ of a decentralized exchange by Figure **??**. The functionality $\mathcal{F}_{\mathsf{DEX}}$ is defined in the $\mathcal{F}_{\mathsf{Ledger}}$-hybrid model and relies on the ledger time and public state to enforce deadlines and asset transfers. Time is modeled via global deadlines $\Delta t_1 < \Delta t_2$, enforced by $\mathcal{F}_{\mathsf{Ledger}}$. If the functionality outputs $\mathsf{success}$, the exchange completes fairly; otherwise, the exchange is aborted.

Ideal Functionality $\mathcal{F}_{\mathsf{DEX}}$

The ideal functionality $\mathcal{F}_{\mathsf{DEX}}$ operates in the $\mathcal{F}_{\mathsf{Ledger}}$-hybrid model. It is parameterized by time bounds $\Delta t_1 < \Delta t_2$, and interacts with $\mathcal{F}_{\mathsf{Ledger}}$ to enforce time, store state, and manage asset escrow. Denote $\mathcal{A}$ as the adversary controlling corrupted parties. Upon activation, $\mathcal{F}_{\mathsf{DEX}}$ behaves as follows:

- **Register:** Upon receiving $(\mathsf{register}, \mathsf{pid}, \mathsf{pk}_x)$ from a party x, stores pk_x in its local state via $\mathcal{F}_{\mathsf{Ledger}}$. Sends the message the $\mathcal{A}$.
- **Deposit:** Upon receiving $(\mathsf{freeze}, \mathsf{Token}, v)$, instructs $\mathcal{F}_{\mathsf{Ledger}}$ to lock the tokens in escrow. Withdrawals are permitted only if the exchange either does not commence or terminates in $\mathsf{failure}$.
- **Commit:** Upon receiving valid commitment $(\mathsf{commit}, \mathsf{pk}_A, C^{(A)})$ (resp. $C^{(B)}$) from Alice (resp. Bob), stores it after verify validity of the commitment. Start the timer when both parties have submitted the commitments.
- **Reveal:** Upon receiving (reveal, K_x) (x is Alice or Bob) before Δt_1, stores the message on $\mathcal{F}_{\mathsf{Ledger}}$ if it is valid. Proceed to **Determine** if both K_A and K_B are correctly provided. Sends the message the $\mathcal{A}$.
- **Complain:** At time Δt_1, if exactly one key is revealed, allow the honest party to complain to the watchers, each of whom will respond with their respective K_i through $\mathcal{F}_{\mathsf{Ledger}}$.
- **Determine:** At any time before Δt_2:
  - If both keys are revealed, instructs $\mathcal{F}_{\mathsf{Ledger}}$ to transfer tokens and output $(\mathsf{success})$.
  - If one key and $\geq t$ watcher keys are revealed, does the same as above.
  - Otherwise, at Δt_2, instructs $\mathcal{F}_{\mathsf{Ledger}}$ to refund and output $(\mathsf{failure})$.

Fig. 12: The ideal functionality $\mathcal{F}_{DEX}$

Real Protocol Π_{DEX}

Any two exchangers (Alice and Bob) leverage the same access structure $\mathbb{A} = (2 \text{ of } (\mathsf{Alice}, \mathsf{Bob}, (t \text{ of } (W_1, \ldots, W_n))))$ (shown by Figure **??**). The real-world protocol Π_{DEX} is executed on smart contracts and implements $\mathcal{F}_{\mathsf{DEX}}$ via PVGSS.

- **Register:** Any entity registers its public key pk_x via $(\mathsf{register}, \mathsf{id}, \mathsf{pk}_x)$.
- **Deposit:** Alice and Bob freeze tokens via $(\mathsf{freeze}, \mathsf{pk}_A, \mathsf{Token}_A, v_A)$ and $(\mathsf{freeze}, \mathsf{pk}_B, \mathsf{Token}_B, v_B)$, respectively. If the session later terminates in `failure`, either party may reclaim its tokens via $(\mathsf{withdraw}, \mathsf{id})$.
- **Commit (swap1):** Bob (resp. Alice) invokes $(\mathsf{swap1}, \mathsf{id}, \{C^{(B)}\}, \mathsf{prfs}^B)$ where $(C^{(B)}, \mathsf{prfs}^B) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}, pks)$, $pks = \{\{\mathsf{pk}_i\}_{i\in[1,n]}, \mathsf{pk}_A, \mathsf{pk}_B\}$ and $C^{(B)} = \{\{C_i^{(B)}\}, C_A^{(B)}, C_B^{(B)}\}$. The contract runs $\mathsf{PVGSSVerify}(C^{(B)}, \mathsf{prfs}^B, \mathbb{A})$; if it fails, the transaction is aborted.
- **Reveal (swap2):** Alice (resp. Bob) invokes $(\mathsf{swap2}, \mathsf{id}, K_A, \mathsf{prf}_{K_A})$ where $(K_A, \mathsf{prf}_{K_A}) \leftarrow \mathsf{PVGSSPreRecon}(C_A^{(A)}, \mathsf{sk}_A)$. The contract checks $\mathsf{PVGSSKeyVrf}(C_A^{(A)}, K_A, \mathsf{prf}_{K_A}, pk_A)$; if valid, K_A is stored and Bob fetches it to recover Alice's secret $g^{s_A} \leftarrow \mathsf{PVGSSRecon}(\mathbb{A}, Q)$, where $Q = \{K_A, K_B\}$ and K_B is generated by Bob itself.
- **Complain:** If at time $t > \Delta t_1$, one party (say Alice) has not revealed, the other (Bob) may call $(\mathsf{complain}, \mathsf{id})$, triggering a notification to all watchers to execute $(K_i, \mathsf{prf}_{K_i}) \leftarrow \mathsf{PVGSSPreRecon}(C_i^{(A)}, \mathsf{sk}_i)$ and publish $(K_i, \mathsf{prf}_{K_i})$ via `swap2`.
- **Determine:** Any user may call $(\mathsf{determine}, \mathsf{id})$ before Δt_2. The contract checks:
  - Both exchangers revealed $\Rightarrow$ `success`.
  - One exchanger + $\geq t$ watchers revealed $\Rightarrow$ `success`.
  - Otherwise $\Rightarrow$ after Δt_2, set `failure` and refund.

Fig. 13: The real-world protocol Π_{DEX} for implementing $\mathcal{F}_{DEX}$

Real-world implementation. Figure **??** describes the real-world protocol Π_{DEX}, which securely implements $\mathcal{F}_{\mathsf{DEX}}$ in the $\mathcal{F}_{\mathsf{Ledger}}$-hybrid model. Each execution instance is uniquely identified by a session id, which binds all operations (**register**, **deposit**, etc.) to a single atomic exchange and prevents cross-session interference. The protocol involves two exchangers (Alice and Bob) and n watchers, who are randomly selected from a registered pool for each id. A state variable tracks the lifecycle of the instance. Without loss of generality, Alice offers v_1 units of Token_1 in exchange for v_2 units of Token_2.[8]

Incentive Mechanism A swap instance ends with either $\mathsf{success}$ or $\mathsf{failure}$ definitely (proved by Lemma **??**). Specifically, the $\mathsf{success}$ can be determined before Δt_2 and the $\mathsf{failure}$ is determined only after Δt_2. We assume that all

[8] We assume rational exchangers with access to external price oracles.

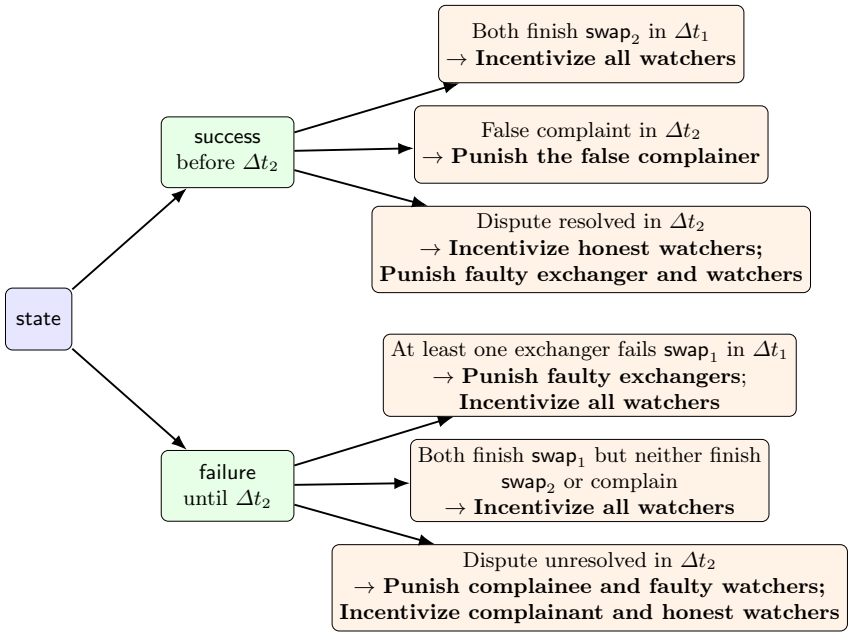

Fig. 14: The incentive and penalty mechanism

entities are required to make stakes in the system; and the number of swap instances a watcher can serve simultaneously is proportional to the amount of its stakes. Figure **??** presents the mindmap to illustrate all cases of results and the corresponding incentive policy a swap instance.

The success results include three cases. The first case is optimistic scenario where both exchangers honestly invoke swap_1 and swap_2 before Δt_1. In this case, the watchers are not involved and each of them is rewarded with a small tip. The second case introduces the scenario where some exchanger may propose false complaint. In this case, the complainer will be punished. The third case shows the scenario where one exchanger does not execute swap_2 before Δt_2. Then, the other exchanger notice the n watchers to resolve the dispute. In this case, faulty exchanger and watchers are punished to reward the honest watchers who send swap_2 before Δt_2.

The failure results also contains three cases. The first case means at least one exchangers give up in Δt_1. Hence, the faulty exchanger will be punished and all watchers are rewarded. The second case means both exchangers negotiate to abort the swap instance before Δt_2, then all watchers are rewarded. The third case is an unexpected scenario, where honest complaint is raised but unresolved due to insufficient honest watchers. We impose penalties on the complainee and faulty watchers to compensate the complainant and honest watchers.

6.3 Security Properties

We prove in Appendix **??** that the real-world protocol Π_{DEX} UC-securely realizes the ideal functionality $\mathcal{F}_{\mathsf{DEX}}$ and the DEX Π_{DEX} guarantees the security properties by Lemmas **??**-**??**.

7 Experimental Evaluation

We implement our PVGSS scheme using Golang (off-chain) and Solidity (on-chain). The method in [**?**] supports only boolean formula-based access structures; therefore, we adopt the approach proposed in [**?**] to transform a general access structure into a LSSS matrix. To make on-chain and off-chain operations compatible, we choose BN128 as the asymmetric elliptic curve group, which is officially supported by Ethereum. The benchmarks of PVGSS schemes are executed on a Intel (R) Core (TM) i7-11800H @2.30GHz with 4GB RAM running Ubuntu 22.04.5 LTS and Golang1.22.0. The smart contract codes are deployed and tested on Ethereum Sepolia testnet.

We choose the access structure designed for the DEX (cf. Figure **??**) to evaluate the performance of the PVGSS scheme, where $t = n/2+1$ and n ranges from 100 to 1000. Figure **??**-**??** show the off-chain computational overheads of the PVGSS schemes. The cost of PVGSSShare and PVGSSRecon are super-linear, due to polynomial evaluation and Lagrange interpolation in Shamir SS-based approach and matrix multiplication in LSSS-based approach. Besides, the PVGSSPreRecon algorithm costs about 418μs for calculation of each K_i. It can be seen all algorithms can be calculated within seconds given $n = 1000$, indicating practicality of the PVGSS schemes in large-scale systems.

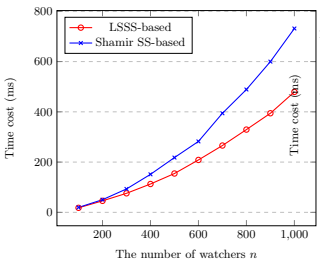

Fig. 15: Cost of PVGSSShare

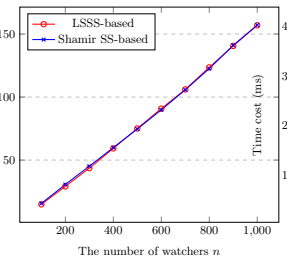

Fig. 16: Cost of PVGSSVerify

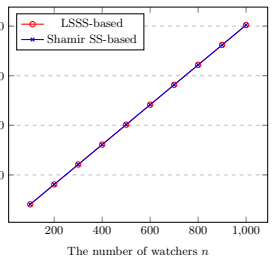

Fig. 17: Cost of n times of PVGSSKeyVrf

Further, we evaluate the gas consumption of the DEX. As proved in Lemma **??**, we can set $t = 1$ and arbitrary $n \geq 1$ for the DEX in practice. We present the on-chain gas cost with n ranging from 1 to 10. We first introduce the cost in the normal case, where both exchangers honestly abide by the protocol. In swap_1, calculation of PVGSSVerify and storage of $\{C_i\}$ are included. The swap_2, which

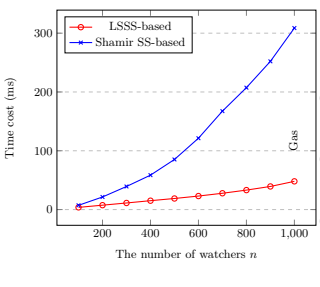

Fig. 18: PVGSSRecon with an authorized set size $t+1$

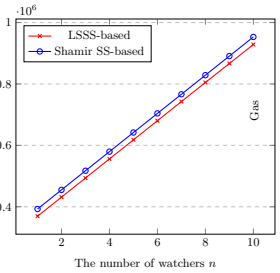

Fig. 19: On-chain gas cost of $\mathsf{swap}_1 + \mathsf{swap}_2$

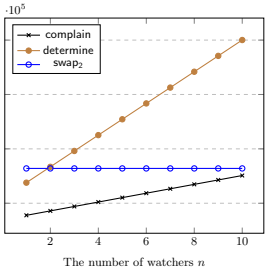

Fig. 20: On-chain cost of complain, determine, swap_2

invokes PVGSSKeyVrf, roughly costs a constant amount gas (i.e., 82020) for each exchanger. The PVGSSKeyVrf algorithm is the same for both LSSS-based and Shamir SS-based PVGSS, hence they cost equal gas consumption. Figure **??** depicts the gas consumption of the $\mathsf{swap}_1 + \mathsf{swap}_2$ on smart contract. It can be observed that the LSSS-based PVGSSShare is slightly more efficient, as matrix multiplication is somewhat more computationally intensive than Lagrange interpolation in Solidity. Then, we depict the gas cost in exceptional scenarios where arbitration procedures are required. Particularly, an exchanger calls complain, then each watcher invokes swap_2 and finally anyone can execute determine. Figure **??** shows the gas cost of functionalities of complain, determine and swap_2.

Table 3: Monetary cost of the on-chain operations

Operation	Gas cost	USD
register	167767	\$0.102
freeze	71250	\$0.043
swap_1	n=1 310787	\$0.189
	n=5 559855	\$0.341
	n=9 808882	\$0.493
swap_2	82020	\$0.05
total, i.e., $\mathsf{swap}_1 + \mathsf{swap}_2$ (Shamir SS-based)	n=1 392807	\$0.239
	n=5 813639	\$0.391
	n=9 1062603	\$0.543
complain	n=1 38894	\$0.024
	n=5 55166	\$0.034
	n=9 71414	\$0.044
determine	n=1 68917	\$0.042
	n=5 127209	\$0.078
	n=9 185499	\$0.113

As of the time of writing (November 25, 2025), the median gas price is 0.206 gwei, and the price of ETH is 2,958 USD.[9] Table **??** presents the monetary cost of each operation on the smart contract. For convenience, only the Shamir SS-based PVGSS is evaluated. The gray cells highlight the optimistic case, where the cost for each exchanger is only \$0.239/\$0.391/\$0.543 for $n = 1/5/9$ and $t = 1$. If watchers are involved in the pessimistic case, the complaint cost is less than 0.044 USD, and each watcher incurs a cost of approximately 0.05 USD (as swap_2 requires).

8 Conclusion

We begin by introducing the concept of publicly verifiable generalized secret sharing (PVGSS) and compare it with other SS-related schemes (i.e., LSSS, GSS, VSS, PVSS and CPABE). The comparison highlights that PVGSS not only supports generalized access structure, but also facilitates public verifiability, making it ideal primitive for transparent systems like blockchain. Then we propose two constructions with unified description: one is Shamir-based and the other is LSSS-based. Rigorous proofs are given to prove the security proofs of the constructions. Furthermore, we build a PVGSS-based decentralized exchange (DEX), which guarantees optimistic fairness and passive arbitration due to the sophisticated designed access structure. Comprehensive experiments and detailed analysis demonstrates the feasibility of the DEX in real-world applications. Looking ahead, we plan to further explore PVGSS-related research. First, we aim to implement additional applications discussed in Section **??** and beyond. Second, we intend to investigate more PVGSS constructions. Lastly, we will explore variations of PVGSS schemes, such as proactive PVGSS or q-PVGSS, akin to proactive SS [**?**] and q-PVSS [**?**].

A UC-Based Security of the DEX

In this appendix, we prove the security of the decentralized exchange protocol Π_{DEX} in the Universal Composability (UC) framework [**?**]. Specifically, we show that Π_{DEX} securely realizes the ideal functionality $\mathcal{F}_{\mathsf{DEX}}$ under static corruptions, assuming the underlying PVGSS construction satisfies correctness, public verifiability, and IND1-secrecy (Theorems **??** –**??**).

Blockchain Model. We work in the UC hybrid model with a global ledger functionality $\mathcal{F}_{\mathsf{Ledger}}$, which maintains an append-only public state and executes smart contracts deterministically. The adversary controls transaction scheduling and message delivery, but cannot modify ledger state or contract code. This abstraction is standard in UC analyses of blockchain-based protocols. The global ledger functionality $\mathcal{F}_{\mathsf{Ledger}}$ maintains a public, append-only state.

[9] `https://etherscan.io/gastracker`

Adversarial Model. We consider a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that statically corrupts a subset of parties before protocol execution. The environment $\mathcal{Z}$ observes all public ledger events and interacts arbitrarily with $\mathcal{A}$.

Proof System Assumptions. We assume that the NIZK proofs used in $\mathsf{PVGSSShare}$ are simulation-extractable in the Random Oracle Model (ROM). In particular, for any accepting proof generated by a corrupted party, there exists a PPT extractor that recovers a witness consistent with the statement. The DLEQ proofs used in $\mathsf{swap2}$ satisfy soundness and adaptive zero-knowledge.

Theorem 5 (UC Security of Π_{DEX}). *Assume the underlying PVGSS scheme satisfies correctness, public verifiability, and IND1-secrecy (Theorems* **??**–**??***). Then, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ in the real world interacting with Π_{DEX} and $\mathcal{F}_{\mathsf{Ledger}}$, there exists a PPT simulator $\mathcal{S}$ in the ideal world interacting only with $\mathcal{F}_{\mathsf{DEX}}$ and $\mathcal{F}_{\mathsf{Ledger}}$, such that for all PPT environments $\mathcal{Z}$,*

$$\mathsf{EXEC}_{\Pi_{\mathsf{DEX}},\mathcal{A},\mathcal{Z}} \approx_c \mathsf{EXEC}_{\mathcal{F}_{\mathsf{DEX}},\mathcal{S},\mathcal{Z}},$$

where $\approx_c$ denotes computational indistinguishability.

A.1 Simulator Construction

We construct a simulator $\mathcal{S}$ that interacts with $\mathcal{F}_{\mathsf{DEX}}$ and $\mathcal{F}_{\mathsf{Ledger}}$, while internally simulating the execution of Π_{DEX} for $\mathcal{A}$. The simulator internally emulates all interactions with $\mathcal{F}_{\mathsf{Ledger}}$, ensuring that the interface exposed to the environment is indistinguishable from that of the real ledger.

Register. For each honest party, $\mathcal{S}$ samples a key pair $(sk_i, pk_i) \leftarrow \mathsf{PVGSSSetup}(\lambda)$ and registers pk_i with $\mathcal{F}_{\mathsf{DEX}}$. For corrupted parties, $\mathcal{S}$ forwards the public keys chosen by $\mathcal{A}$. This distribution matches the real protocol.

Deposit. Valid asset-freezing transactions submitted by $\mathcal{A}$ are forwarded to $\mathcal{F}_{\mathsf{DEX}}$ via $\mathcal{F}_{\mathsf{Ledger}}$. Invalid requests are rejected, exactly as in the real-world execution.

Commit (swap1). Upon receiving a commitment message ($\mathsf{swap1}$, $id, \{C_i\}, \pi_s$):

- *from corrupted exchanger*, $\mathcal{S}$ invokes the knowledge extractor guaranteed by the simulation-extractability of the NIZK proof system used in $\mathsf{PVGSSShare}$ (see Section **??** and Theorem **??**). If extraction fails, $\mathcal{S}$ simulates contract rejection. Otherwise, $\mathcal{S}$ forwards the extracted information to $\mathcal{F}_{\mathsf{DEX}}$.
- *from each honest party i*, $\mathcal{S}$ samples internal exponents $\alpha_i \xleftarrow{R} \mathbb{Z}_q$ and computes simulated ciphertexts $C_i = \mathsf{pk}_i^0 \cdot g^{\alpha_i} = g^{\alpha_i}$. The simulator then invokes the zero-knowledge simulator $\mathcal{S}_{\mathrm{PVGSS}}$ for the PVGSS NIZK proof system to generate $\pi_s \leftarrow \mathcal{S}_{\mathrm{PVGSS}}(\{C_i\}, \mathbb{A})$. By the zero-knowledge property of the underlying proof system, the tuple $(\{C_i\}, \pi_s)$ is computationally indistinguishable from a valid commitment generated by an honest dealer in the real protocol.

Reveal (swap2). When a decrypted share (K_i, π_{K_i}) is submitted:

- *For corrupted parties*, $\mathcal{S}$ verifies correctness using $\mathsf{PVGSSKeyVrf}$ and updates the state of $\mathcal{F}_{\mathsf{DEX}}$ accordingly.
- *For honest party* i, $\mathcal{S}$ computes $K_i = g^{\alpha_i}$ where $\alpha_i \in \mathbb{Z}_q$ is the internal exponent previously used to generate the simulated commitment $C_i = \mathsf{pk}_i^0 \cdot g^{\alpha_i}$ during the **commit (swap1)** phase. Subsequently, $\mathcal{S}$ invokes the adaptive zero-knowledge simulator $\mathcal{S}_{\mathrm{DLEQ}}$ for the DLEQ proof system to generate $\pi_{K_i} \leftarrow \mathcal{S}_{\mathrm{DLEQ}}(C_i, K_i, g, \mathsf{pk}_i, \alpha_i)$, demonstrating algebraic consistency between C_i and K_i without knowledge of any actual protocol secret share.

Complain. If $\mathcal{A}$ invokes a complaint after deadline Δt_1, $\mathcal{S}$ forwards it to $\mathcal{F}_{\mathsf{DEX}}$.

Determine. Upon determination, $\mathcal{S}$ queries $\mathcal{F}_{\mathsf{DEX}}$ and reproduces the outcome ($\mathsf{success}$ or $\mathsf{failure}$) on the simulated ledger, including asset transfers and penalties.

Theorem 6 (Simulation Consistency). *The simulator maintains internal state to ensure algebraic consistency between simulated commitments and simulated reveals. In particular, the PVGSS encryption scheme supports programmable ciphertexts, ensuring that simulated ciphertexts in* ***commit (swap1)*** *phase and simulated decrypted shares in* ***Reveal (swap2)*** *remain mutually consistent.*

A.2 Indistinguishability Argument

All information observable by the environment $\mathcal{Z}$ is either derived from the public ledger state or from messages sent by corrupted parties. By Theorem **??**, the simulator $\mathcal{S}$ exactly emulates the ledger interface, smart contract logic, and the behavior of corrupted parties. Hence, any difference between the real and ideal executions can only arise from the simulation of honest-party cryptographic messages. We define a sequence of hybrid games to bound this difference.

- **Hybrid 0 (Real):** The actual execution of Π_{DEX} with $\mathcal{A}$.
- **Hybrid 1 (Simulation of Honest Proofs):** Replace real NIZK proofs (PVGSSShare and DLEQ) generated by honest parties with simulated proofs. *Indistinguishability:* Follows from the **Zero-Knowledge** property of the underlying Sigma protocols and DLEQ proofs.
- **Hybrid 2 (Simulation of Ciphertexts):** Replace honest commitments $C = \mathrm{Enc}(pk, s)$ with $C^* = \mathrm{Enc}(pk, 0)$. *Indistinguishability:* Follows from the **IND1-secrecy** of PVGSS (Theorem **??**). Since $\mathcal{A}$ controls an unauthorized set during the commitment phase, the ciphertexts reveal no information about the secret.
- **Hybrid 3 (Extraction):** For corrupted parties, $\mathcal{S}$ extracts witnesses using the ROM programmability. *Indistinguishability:* Follows from the **Soundness** of the NIZK proofs. If $\mathcal{A}$ produces a valid proof without a valid witness, it breaks soundness.

– **Hybrid 4 (Ideal):** $\mathcal{S}$ uses the extracted/simulated values to interact with $\mathcal{F}_{DEX}$. The ledger state is updated exactly as $\mathcal{F}_{DEX}$ dictates.

Summarily, the simulated execution is computationally indistinguishable from the real one due to the following properties:

– **Zero-knowledge simulation.** The underlying NIZK proof systems—namely (i) the Sigma protocol with Fiat–Shamir transformation used in PVGSSShare, and (ii) the DLEQ proof used in PVGSSPreRecon—admit efficient simulators. This allows $\mathcal{S}$ to generate commitments and reveal messages for honest parties that are computationally indistinguishable from those in a real execution, while maintaining algebraic consistency.
– **IND1-secrecy of PVGSS.** By Theorem **??**, the ciphertexts generated in the commitment phase are indistinguishable from encryptions of any other value to parties outside the authorized access structure. This ensures that the simulated ciphertexts used by $\mathcal{S}$ cannot be distinguished from real ones by $\mathcal{A}$ or $\mathcal{Z}$.
– **Soundness of verification.** The soundness of the PVGSS verification algorithms (Theorems **??** and **??**) guarantees that any malformed commitments or decrypted shares submitted by $\mathcal{A}$ are rejected with the same probability in both the real and ideal executions, ensuring identical acceptance and rejection behavior.

A standard hybrid argument over these components implies that the joint view of $\mathcal{Z}$ and $\mathcal{A}$ in the simulated execution is computationally indistinguishable from that in the real execution.

A.3 Security Properties of the DEX

Lemma 3 (Decentralization). *Protocol Π_{DEX} is fully decentralized and does not rely on any trusted third party.*

Proof. Any party can permissionlessly register on-chain as an exchanger or a watcher by depositing the required assets. All protocol logic—including commitment, verification, secret reconstruction, arbitration, and settlement—is executed by smart contracts together with publicly verifiable cryptographic algorithms. No trusted coordinator, manager, or key holder is assumed at any stage of the protocol. Hence, Π_{DEX} is decentralized by design.

Lemma 4 (Termination). *Every execution of Π_{DEX} terminates within a bounded time.*

Proof. The protocol enforces two explicit deadlines $\Delta t_1 < \Delta t_2$. If the required secrets are reconstructed before Δt_1, the session terminates successfully. Otherwise, the smart contract deterministically finalizes the session at time Δt_2 and refunds the frozen assets. This behavior exactly matches the termination semantics of $\mathcal{F}_{\mathsf{DEX}}$, guaranteeing termination regardless of adversarial behavior.

Lemma 5 (Optimistic Fairness). *Protocol Π_{DEX} guarantees fairness: an honest exchanger either obtains the correct counterparty assets or is refunded.*

Proof. Optimistic fairness follows from the following facts.

- *Binding commitments.* Before any asset transfer, both exchangers must publish PVGSS commitments via PVGSSShare, which are publicly validated by PVGSSVerify. Invalid commitments are rejected on-chain.
- *Controlled secret release.* An honest exchanger can learn the counterparty's secret only if the counterparty reveals it honestly, or if at least t watchers reconstruct the missing secret via PVGSSRecon. By the correctness of PVGSS (Theorem **??**), any reconstructed secret is valid, triggering a successful atomic exchange.
- *Detectable deviations.* Invalid encrypted shares in swap1 are rejected by PVGSSVerify (Theorem **??**), and invalid decrypted shares in swap2 are rejected by PVGSSKeyVrf (Theorem **??**). If arbitration fails, dishonest parties are penalized and honest exchangers are refunded.

Therefore, an honest exchanger is never worse off.

Lemma 6 (Accountable Exchangers). *Any dishonest behavior by an exchanger is publicly detectable.*

Proof. In the commitment phase, exchanger accountability is enforced by the public verifiability of encrypted shares via PVGSSVerify (Theorem **??**). In the reveal phase, correctness of decrypted shares is enforced by PVGSSKeyVrf and the soundness of the underlying DLEQ proofs (Theorem **??**). Hence, exchangers cannot deviate without detection.

Lemma 7 (Accountable Arbitration). *All arbitration actions performed by watchers are publicly verifiable and accountable.*

Proof. Upon a complaint, watchers submit decrypted shares together with verifiable proofs generated by PVGSSPreRecon. These submissions are publicly checked by PVGSSKeyVrf, ensuring that any incorrect arbitration behavior is detected.

Lemma 8 (Optimistic and Stateless Arbitration). *Protocol Π_{DEX} supports optimistic and stateless arbitration.*

Proof. If both exchangers are honest, the access structure guarantees that secrets can be reconstructed before Δt_1 without involving watchers, demonstrating optimism. Watchers participate only upon explicit complaints and rely solely on on-chain information, maintaining no local state for individual sessions.

Lemma 9 (Incentive Compatibility). *Π_{DEX} admits incentive-compatible mechanisms that encourage honest behavior.*

Proof. By Lemmas **??** and **??**, all deviations are detectable. This enables the protocol to reward honest behavior and penalize misbehavior through deposits and slashing, making honest participation a rational strategy.

Given the above lemmas, we argue that the following potential threats to the DEX have minimal impact.

Claim 1 (Dos attack) *Any DoS attack resulting in* failure *termination can be detected (by Lemma* **??** *and Lemma* **??***), and the perpetrator will be penalized to compensate the victim using its deposited tokens.* □

Claim 2 (Collusion/Coercion attack) *Owing to the transparency, availability, public accountability of smart contracts, an exchanger cannot obtain any advantage or compromise fairness, even when colluding with watchers.* □

Claim 3 (Sybil attack) *An arbitrageur can be selected as a watcher in proportion to the amount of currency they have deposited. This implies that the nothing-at-stake attack is also eliminated. If a swap instance is ongoing, a proportional amount of deposited currency is locked, rendering a Sybil attack meaningless.* □

Claim 4 (Rushing attack) *Rushing watchers, who attempt to gain benefits by providing arbitration without assistance from exchangers, make a futile attempt, as the honest exchangers incur no loss when following the protocol correctly.* □

B Decisional Diffie-Hellman (DDH) Assumption

The Decisional Diffie-Hellman (DDH) [**?**] assumption states that (g, g^a, g^b, g^{ab}) and (g, g^a, g^b, g^c) are computationally indistinguishable, where $a, b, c \xleftarrow{R} \mathbb{Z}_q$ are chosen uniformly at random. Formally, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ in distinguishing between the two distributions is negligible:

$$\left|\Pr\left[\mathcal{A}(g, g^a, g^b, g^{ab}) = 1\right] - \Pr\left[\mathcal{A}(g, g^a, g^b, g^c) = 1\right]\right| \leq \mathsf{negl}(\lambda),$$

where λ is the security parameter, and $\mathsf{negl}(\lambda)$ denotes a negligible function in λ.

C Formal Definition of Security Properties for PVGSS

Definition 9 (Correctness). *The PVGSS is correct if for all authorized set Q of $\mathbb{A}$, each s in the secret space,*

$$\Pr\left[\begin{array}{l}\mathsf{PVGSSVerify}\left(\{C_i\}, \mathsf{prf}_s, \mathbb{A}\right) = 1 \wedge \\ \mathsf{PVGSSKeyVrf}(C_i, S_i, \mathsf{prf}_{S_i}, \mathsf{pk}_i) = 1, \forall i \in I_Q \wedge \\ S' = g^s\end{array} \middle| \begin{array}{l}(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{PVGSSSetup}(\lambda); \\ (\{C_i\}, \mathsf{prf}_s) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}, \{pk_i\}) \\ \forall i \in I_Q, (S_i, \mathsf{prf}_{S_i}) \leftarrow \mathsf{PVGSSPreRecon}(C_i, \mathsf{sk}_i) \\ (S') \leftarrow \mathsf{PVGSSRecon}(\mathbb{A}, Q);\end{array}\right] = 1$$

Definition 10 (Verifiability of Encrypted Shares). *The PVGSS satisfies verifiability of encrypted shares if for every PPT $\mathcal{A}$,*

$$\Pr\left[\begin{array}{l}\mathsf{PVGSSVerify}\left(\{C_i\}, \mathsf{prf}_s, \mathbb{A}\right) = 1 \wedge \\ \nexists s \in S \ s.t. \ (\{C_i\}_{i\in[n]}, *) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}, \{\mathsf{pk}_i\})\end{array} \middle| \begin{array}{l}(\{\mathsf{pk}_i\}, \{\mathsf{sk}_i\}) \leftarrow \mathsf{PVGSSSetup}(\lambda), \\ (\{C_i\}, \mathsf{prf}_s, \mathbb{A}) \leftarrow \mathcal{A}(\{\mathsf{pk}_i\})\end{array}\right] \leq negl(\lambda)$$

Definition 11 (Verifiability of Share Decryption). *The PVGSS satisfies verifiability of share decryption if for every PPT $\mathcal{A}$,*

$$\Pr\left[\begin{array}{l}\mathsf{PVGSSKeyVrf}(C, S, \mathsf{prf}_S, pk) = 1 \wedge \\ \nexists sk \in \{\mathsf{sk}_i\} \ s.t. \ (S, \mathsf{prf}_S) \leftarrow \mathsf{PVGSSPreRecon}(C, sk)\end{array} \middle| \begin{array}{l}(\{\mathsf{pk}_i\}, \{\mathsf{sk}_i\}) \leftarrow \mathsf{PVGSSSetup}(\lambda), \\ (pk, C, S) \leftarrow \mathcal{A}(\{\mathsf{pk}_i\}), \ pk \in \{\mathsf{pk}_i\}\end{array}\right] \leq negl(\lambda)$$

Definition 12 (IND1-secrecy). ***IND1-secrecy*** *standards for indistinguishability of secrets. PVGSS achieves* ***IND1-secrecy*** *if for any polynomial time adversary $\mathcal{A}$ without an authorized set, $\mathcal{A}$ win the following game played against a challenger with negligible advantage.*

1. *The challenger initiates the* $\mathsf{PVGSSSetup}$ *algorithm for all honest parties and publishes the public keys $\{\mathsf{pk}_i\}$.*
2. *$\mathcal{A}$ generates a challenge access structure $\mathbb{A}^*$. $\mathcal{A}$ chooses a maximum set of indexes $U \subset [1, \cdots, |\mathbb{A}^*|]$, where the shares corresponding to U do not satisfy the access structure $\mathbb{A}^*$. It sends $\mathbb{A}^*, U$ to the challenger.*

3. *The challenger randomly selects values x_0 and x_1 from the secret space. It then randomly selects $b \in \{0,1\}$. Further, it invokes $(\{C_i^*\}, \mathsf{prf}_{x_0}) \leftarrow \mathsf{PVGSSShare}(\mathbb{A}^*, \{\mathsf{pk}_i\})$, which indicates that x_0 is used as the secret. The challenger sends $(\{C_i^*\}, \mathsf{prf}_{x_0}, x_b)$ to $\mathcal{A}$.*
4. *$\mathcal{A}$ then outputs a guess $b' \in \{0,1\}$.*

The $\mathcal{A}$'s advantage in the game is defined as $|Pr[b = b'] - \frac{1}{2}|$.

References

1. Shamir, A. How to share a secret. *Comm. of the ACM*, 1979, 22(11), pp. 612-613.
2. Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L. and Ren, L. Practical asynchronous distributed key generation. In *S&P*, 2022, pp. 2518-2534.
3. Choi, K., Manoj, A. and Bonneau, J. Sok: Distributed randomness beacons. In *S&P*, 2023, pp. 75-92.
4. Syta, E., Jovanovic, P., Kogias, E.K., Gailly, N., Gasser, L., Khoffi, I., Fischer, M.J. and Ford, B. Scalable bias-resistant distributed randomness.In *S&P*,2017, pp. 444-460.
5. Schindler, P., Judmayer, A., Stifter, N. and Weippl, E., 2020, May. Hydrand: Efficient continuous distributed randomness. In 2020 IEEE Symposium on Security and Privacy (SP) (pp. 73-89). IEEE.
6. Sahai, A., and Waters, B. Fuzzy identity-based encryption. In *Eurocrypt*, 2005, pp. 457-473.
7. Karchmer, M. and Wigderson, A. On span programs. In *Annual Structure in Complexity Theory Conference*, 1993, pp. 102-111.
8. Feldman, P. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, 1987, pp. 427-438.
9. Zhang, Z., Li, W., Guo, Y., Shi, K., Chow, S.S., Liu, X. and Dong, J. Fast RS-IOP Multivariate Polynomial Commitments and Verifiable Secret Sharing. In *USENIX Security*, 2024, pp. 3187-3204.
10. Cascudo, I., and David, B. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In *Eurocrypt*, 2024, pp. 216-248.
11. Cascudo, I., and David, B. SCRAPE: Scalable randomness attested by public entities. In *ACNS*, 2017, pp. 537-556.
12. Kate, A., Zaverucha, G.M. and Goldberg, I., Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.
13. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S. and Stern, G. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In *Crypto*, 2023, pp. 39-70.
14. Stadler, M. Publicly verifiable secret sharing. In *Eurocrypt*, 1996, pp. 190-199.
15. Ito, M., Saito, A. and Nishizeki, T. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan*, 1989, 72(9), pp. 56-64.
16. Benaloh, J., and Leichter, J. Generalized secret sharing and monotone functions. In *Crypto*, 1990.
17. Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M. and Zhang, Y. Cryptography with weights: MPC, encryption and signatures. In *CRYPTO*, 2023, pp. 295-327.
18. Cao, B., Xiao, S., Shi, L., Wang, T., Chen, J., Wang, J., Ling, X., Xu, H., Zhang, S. and Liu, E. Web 3.0: A survey on the architectures, enabling technologies, applications, and challenges. *IEEE Communications Surveys & Tutorials*, 2025.

19. Bethencourt, J., Sahai, A., and Waters, B. Ciphertext-Policy Attribute-Based Encryption. In *S&P*, 2007, pp. 321-334.
20. Lewko, A., and Waters, B. Decentralizing attribute-based encryption. In *Eurocrypt*, 2011, pp. 568-588.
21. Lin, C., Hu, H., Chang, C.C. and Tang, S. A publicly verifiable multi-secret sharing scheme with outsourcing secret reconstruction. *IEEE Access*, 2018, 6, pp. 70666-70673.
22. Beimel, A. Secret-Sharing Schemes for General Access Structures: An Introduction. *Cryptology ePrint Archive*. 2025.
23. Schnorr, C.P., 1990. Efficient identification and signatures for smart cards. In *Crypto*, 1990, pp. 239-252.
24. Drăgan, C.C. and Ţiplea, F.L., . Distributive weighted threshold secret sharing schemes. *Information sciences*, 2016, 339, pp. 85-97.
25. Beimel, A., Tassa, T., and Weinreb, E. Characterizing ideal weighted threshold secret sharing. In *TCC*, 2005, pp. 600-619.
26. Chen, Q., Tang, C. and Lin, Z. Efficient explicit constructions of multipartite secret sharing schemes. *IEEE TIT*, 2021, 68(1), pp. 601-631.
27. Tassa, T., and Dyn, N. Multipartite secret sharing by bivariate interpolation. *JoC*, 2009, 22(2), pp. 227-258.
28. Simmons, G.J., 1988, August. How to (really) share a secret. In *Eurocrypt*, 1988, pp. 390-448.
29. Damgård, I. On Σ-protocols. *Lecture Notes, University of Aarhus, Department for Computer Science*, 2002.
30. Fiat, A., and Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *Eurocrypt*, 1986, pp. 186-194.
31. Tassa, T. Hierarchical threshold secret sharing. *JoC*, 2007, 20(2), pp. 237-264.
32. Chattopadhyay, A.K., Saha, S., Nag, A. and Nandi, S. Secret sharing: A comprehensive survey, taxonomy and applications. *Computer Science Review*, 2024, 51, 100608.
33. Beimel, A. Secret-sharing schemes: A survey. In *ICC*, 2011, pp. 11-46.
34. Cascudo, I., David, B., Garms, L., and Konring, A. YOLO YOSO: fast and simple encryption and secret sharing in the YOSO model. In *Asiacrypt*, 2022, pp. 651-680.
35. Cascudo, I., and David, B. ALBATROSS: publicly attestable batched randomness based on secret sharing. In *Asiacrypt*, 2020, pp. 311-341.
36. Fujisaki, E., and Okamoto, T. A practical and provably secure scheme for publicly verifiable secret sharing and its applications. In *Eurocrypt*, 1998, pp. 32-46.
37. Schoenmakers, B. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic. In *Crypto*, 1999, pp. 148-164.
38. Ruiz, A., and Villar, J. L. Publicly verifiable secret sharing from Paillier's cryptosystem. In *SAC*, 2005.
39. Erwig, A., Faust, S., Hostáková, K., Maitra, M. and Riahi, S. Two-party adaptor signatures from identification schemes. In *PKC*, 2021, pp. 451-480.
40. Thyagarajan, S.A., Malavolta, G. and Moreno-Sanchez, P. Universal atomic swaps: Secure exchange of coins across all blockchains. In *S&P*, 2022, pp. 1299-1316.
41. Zhang, L., Kan, H., Qiu, F. and Hao, F. A Publicly Verifiable Optimistic Fair Exchange Protocol Using Decentralized CPABE. *CJ*, 2024, 67(3), pp. 1017-1029.
42. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P. and Riahi, S. Generalized channels from limited blockchain scripts and adaptor signatures. In *Asiacrypt*, 2021, pp. 635-664.

43. Augusto, A., Belchior, R., Correia, M., Vasconcelos, A., Zhang, L. and Hardjono, T. Sok: Security and privacy of blockchain interoperability. In *S&P*, 2024, pp. 3840-3865.
44. Xu, J., Paruch, K., Cousaert, S. and Feng, Y. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Comp. Surv.*, 2023, 55(11), pp. 1-50.
45. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L. and Juels, A. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *S&P*, 2020, pp. 910-927.
46. Baum, C., David, B. and Frederiksen, T.K. P2DEX: privacy-preserving decentralized cryptocurrency exchange. In *ACNS*, 2021, pp. 163-194.
47. Heimbach, L., Schertenleib, E. and Wattenhofer, R. Risks and returns of uniswap v3 liquidity providers. In *AFT*, 2022, pp. 89-101.
48. Park, S., Bastani, O. and Kim, T. ACon2: Adaptive Conformal Consensus for Provable Blockchain Oracles. In *USENIX Security*, 2023, pp. 3313-3330.
49. Xie, T., Zhang, J., Cheng, Z., Zhang, F., Zhang, Y., Jia, Y., Boneh, D. and Song, D. zkbridge: Trustless cross-chain bridges made practical. In *CCS*, 2022, pp. 3003-3017.
50. Li, Y., Liu, H. and Tan, Y. POLYBRIDGE: A crosschain bridge for heterogeneous blockchains. In *ICBC*, 2022, pp. 1-2.
51. Dziembowski, S., Faust, S. and Hostáková, K. General state channel networks. In *CCS*, 2018, pp. 949-966.
52. Tyagi, N., Arun, A., Freitag, C., Wahby, R., Bonneau, J. and Mazières, D. Riggs: Decentralized sealed-bid auctions. In *CCS*, 2023, pp.1227–1241.
53. Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T. and Reyzin, L. Can a public blockchain keep a secret?. In *TCC*, 2020, pp. 260-290.
54. Goyal, V., Kothapalli, A., Masserova, E., Parno, B. and Song, Y. Storing and retrieving secrets on a blockchain. In *PKC*, 2022, pp. 252-282.
55. Cerulli, A., Connolly, A., Neven, G., Preiss, F.S. and Shoup, V. vetkeys: How a blockchain can keep many secrets. *Cryptology ePrint Archive.* 2023.
56. Liu, Z., Cao, Z. and Wong, D.S. Efficient generation of linear secret sharing scheme matrices from threshold access trees. *Cryptology ePrint Archive.* 2010
57. Chaum, D., & Pedersen, T. P. Wallet databases with observers. In *CRYPTO*, 1992, pp. 89-105.
58. Bagherpour, B., 2022. An efficient publicly verifiable and proactive secret sharing scheme. *Adv. in Math. of Comm.*, 16(4), pp. 671-691.
59. Alon, N., Galil, Z. and Yung, M. Efficient dynamic-resharing "verifiable secret sharing" against mobile adversary. In *EPA*, 1995, pp. 523-537.
60. Pagnia, H. and Gärtner, F.C. On the impossibility of fair exchange without a trusted third party. Technical Report TUD-BS-1999-02, Darmstadt University of Technology, Department of Computer Science, Darmstadt, Germany. 1999.
61. Galil, Z. and Yung, M. Partitioned encryption and achieving simultaneity by partitioning. *Infor. Proc. Letters*, 1987, 26(2), pp.81-88.
62. Canetti, R. Security and composition of multiparty cryptographic protocols. *Journal of Cryptography*, 2000, 13(1), pp.143-202.